

eBook for Pizza.owl Ontology Practice using Protégé

- [From Ontology Learning to Executable Knowledge Architecture](#)
- [Why the Pizza.owl Tutorial Matters](#)
- [About This Book](#)
- [Why Ontology Matters Today](#)
- [Ontology Within the EKA Framework](#)
- [This Book Is NOT Only About Pizza](#)
- [Who This Book Is For](#)
- [What Makes This Book Different](#)
- [How to Use This Book](#)
- [Acknowledgements](#)
- [Related Resources](#)
- [Final Thoughts](#)
- [Quick Links](#)

From Ontology Learning to Executable Knowledge Architecture

The Semantic Web has existed for more than two decades, yet for many architects, engineers, analysts, and AI practitioners, ontology engineering still feels abstract, academic, or difficult to approach systematically.

One of the reasons is simple:

Most ontology learning materials focus either too heavily on theory or too narrowly on tool operations.

As a result, many learners can:

- memorize OWL terminology,
- click through Protégé demonstrations,
- or understand isolated semantic concepts,

but still struggle to answer the most important question:

Why does ontology engineering actually matter in modern enterprise and AI systems?

This book was created to answer that question.



Why the Pizza.owl Tutorial Matters

Among all ontology learning materials ever created, Michael DeBellis' Pizza OWL tutorial remains one of the most influential and practical introductions to OWL ontology engineering.

Using a familiar pizza domain, the tutorial teaches learners how to:

- think semantically,
- construct ontology hierarchies,
- model relationships,
- apply OWL logic (rules),
- and understand reasoning behavior inside Protégé

What makes the Pizza tutorial special is that it gradually introduces semantic complexity in a highly structured manner.

Rather than overwhelming learners immediately with advanced ontology theory, it guides readers step by step through the evolution of a real semantic model.

That educational philosophy strongly influenced the structure of this book.

This eBook is based primarily on:

- Michael DeBellis' Protégé Pizza OWL Tutorial
- the Protégé ontology engineering workflow
- and a complete hands-on video series published through my YouTube channel beginning in 2023

About This Book

This eBook was written as a companion knowledge guide to the YouTube playlist:

****Protégé OWL Pizza Tutorial Hands-on Series****
(<https://www.youtube.com/playlist?list=PL6DEHvciXKeUx4P32B3hKMK1t6mC8RhsW>)

Since publishing the Protégé OWL Pizza Tutorial series in 2023, I've realized how valuable practical ontology learning remains for architects and AI practitioners. What started as a hands-on walkthrough of Michael DeBellis' tutorial gradually became an important foundation for my EKA (Executable Knowledge Architecture) thinking — connecting ontology, knowledge graphs, and executable intelligence. The playlist may use pizza as the example domain, but its real purpose is helping learners develop semantic thinking and understand how machine-readable knowledge is engineered.

While, the goal is not merely to transcribe the videos.

Instead, this book expands the tutorial into a much deeper professional learning experience by combining:

- ontology engineering theory,
- hands-on Protégé practice,
- semantic modeling methodology,
- enterprise architectural thinking,
- and modern knowledge graph perspectives.

Every chapter corresponds closely to the learning progression of the original video series so readers can learn both visually and conceptually in parallel.

This synchronization is intentional.

Ontology engineering is best learned progressively.

Why Ontology Matters Today

We are entering a new era of intelligent systems.

Modern enterprises increasingly rely on:

- AI systems,
- semantic search,
- enterprise knowledge graphs,
- digital twins,
- intelligent automation,
- contextual reasoning,
- and machine-readable knowledge.

Traditional architectures built purely on:

- servers and databases,
- documents,
- diagrams
- and APIs

are no longer sufficient for advanced semantic intelligence.

The missing layer here is:

Meaning

Ontology engineering provides that layer.

OWL ontologies allow organizations to formally define:

- concepts,
- relationships,
- constraints,
- semantics,
- and inference logic

in a machine-readable and machine-processable form.

This transforms static information into executable knowledge.

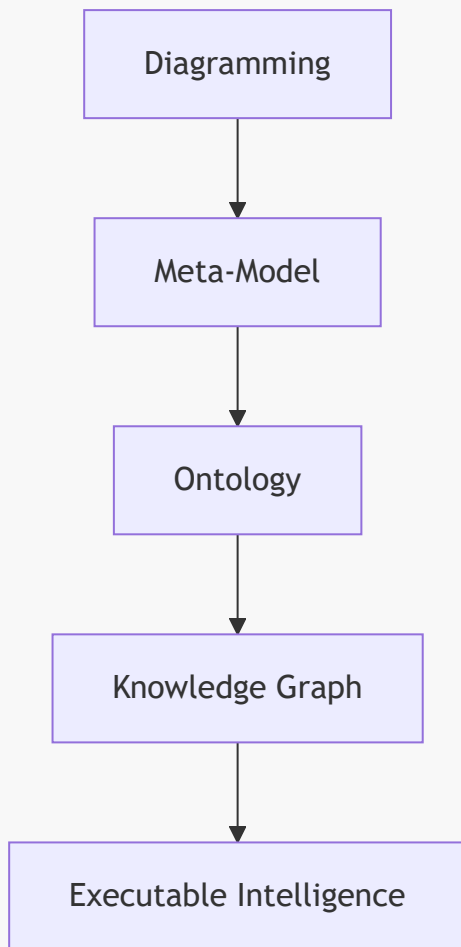
Ontology Within the EKA Framework

This book also introduces ontology engineering through the lens of the EKA (Executable Knowledge Architecture) framework.

EKA represents an architectural approach for transforming enterprise knowledge into executable intelligence systems.

Within EKA, the implementation roadmap is:

The EKA Implementation Roadmap



Ontology occupies the critical semantic transformation layer.

This is where:

- visual architecture,
- conceptual models,
- and enterprise abstractions

become:

- machine-readable semantics,
- reasoning structures,
- and graph-ready knowledge.

Protégé therefore becomes more than just an ontology editor.

It becomes an engineering environment for executable semantics.

This perspective fundamentally changes how ontology engineering should be understood.

This Book Is NOT Only About Pizza

Although the tutorial domain uses pizza examples, the real purpose of the tutorial is much larger.

The Pizza ontology teaches foundational semantic engineering patterns that scale into real-world domains such as:

- enterprise architecture,
- healthcare,
- finance,
- manufacturing,
- government,
- cybersecurity,
- digital twins,
- and AI knowledge systems.

The same semantics principles used to model:

```
(Pizza)-[:hasTopping]->(CheeseTopping)
```

can later scale into:

```
(Application)-[:supportsCapability]->(businessFunction)
```

or:

```
(Device)-[:connectedTo]->(Sensor)
```

Ontology engineering is domain-independent!

The Pizza example simply provides an approachable learning environment.

Who This Book Is For

This book is intended for:

- Enterprise Architects
- Solution Architects
- Data Architects
- Knowledge Engineers
- AI Engineers
- Semantic Web Learners
- Knowledge Graph Practitioners
- Protégé Beginners
- Meta-Modeling Practitioners
- EKA Learners
- Digital Transformation Professionals

No prior ontology experience is required.

However, reader with backgrounds in:

- UML,
- ER Modeling,
- Enterprise Architecture,
- Databases,
- Graph Technologies,
- or Software Engineering

will often recognize important conceptual parallels throughout the book.

What Makes This Book Different

Most ontology books fall into one of two extremes:

Style	Problem
Highly academic	Difficult for practitioners
Pure tool tutorials	Lack conceptual depth

This book intentionally bridges both worlds.

The objectives is to create **A Practical Ontology Engineering Handbook** that remains:

- technically rigorous,
- professionally structured,
- architecturally meaningful,
- and implementation-oriented.

The book therefore combines:

- OWL theory,
- Protégé operations,
- semantic engineering methodology,
- EKA architecture thinking,
- and knowledge graph perspectives

into a single progressive learning journey.

How to Use This Book

The recommended learning approach is:

1. Watch the corresponding video chapter
2. Read the matching eBook chapter carefully
3. Repeat the Protégé exercises manually
4. Reflect on the semantic modeling principles
5. Connect the concepts back to enterprise knowledge systems

Ontology engineering is not mastered through passive reading alone.

It requires semantic thinking practice.

The goal is not merely learning Protégé.

The goal is learning how to engineer machine-understandable meaning.

Acknowledgements

Special thanks to:

- Michael DeBellis for creating one of the most influential practical OWL learning resources available to the ontology community. (Visit Michael's homepage here: <https://www.michaeldebellis.com/>)
- Stanford University and the Protégé community for advancing open ontology engineering tools and Semantic Web technologies.

Related Resources

Official Resources:

- Protégé Official Website: <https://protege.stanford.edu/>
- Protégé Pizza Repository: https://github.com/yasenstar/protege_pizza/tree/main
- EKA Official Website: <https://xiaoqi.com/>

Learning Resources:

- YouTube Channel - Xiaoqi(Yasen) Zhao: <http://www.youtube.com/@yasenzhao>
- Udemy Course - Xiaoqi Zhao: <https://www.udemy.com/user/xiaoqi-zhao>

Final Thoughts

Ontology engineering is NO LONGER a niche academic discipline.

It is rapidly becoming part of the foundational infrastructure for:

- AI systems,
- enterprise knowledge platforms,
- semantic interoperability,
- and executable intelligence architectures.

Understanding ontology today is increasingly similar to understand databases or APIs twenty years ago.

It is becoming a core architectural capability.

The journey begins with a simple Pizza ontology.

But the destination is much larger:

Executable knowledge

WELCOME TO THE WORLD OF ONTOLOGY ENGINEERING.

Quick Links

Here are the quick links to every chapter:

- [Chapter 01 - Entering the World of Ontology Engineering with Protégé and `Pizza.owl`](#)
- [Chapter 02 - Building Your First Ontology in Protégé](#)
- [Chapter 03 - Installing Protégé and Understanding the Ontology Engineering Workspace](#) - WIP

"You" are the learner of this book, so while you're reading, I'll say to "you" directly instead of "learner" / "reader" from now on.

Last updated at: 2026/05/15

Chapter 01 - Entering the World of Ontology Engineering with Protégé and **Pizza.owl**

The journey into ontology engineering often begins with a deceptively simple example: **pizza**. Behind this famous **pizza.owl** tutorial lies one of the most influential learning paths in the Semantic Web and Knowledge Graph ecosystem. In this first chapter, we establish the conceptual foundations of ontology modeling, introduce the Protégé ontology editor, and position ontology engineering within the broader vision of Executable Knowledge Architecture (EKA).

This chapter is based primarily on Michael DeBellis' excellent tutorial [A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins](#), which modernized and extended the original Manchester Pizza Tutorial into a comprehensive hands-on learning experience.

At the same time, this book extends beyond the tutorial itself. The goal here is not only to learn Protégé as a standalone modeling tool, but to understand ontology as a critical layer in the transformation from diagrams to executable intelligence.

- [1.1 Why the Pizza Tutorial Became an Industry Classis](#)
- [1.2 Protégé: More Than an Ontology Editor](#)
- [1.3 The EKA Perspective: From Diagrams to Executable Intelligence](#)
- [1.4 Understanding OWL: The Semantic Core](#)
 - [1.4.1 Classes: Modeling Conceptual Categories](#)
 - [1.4.2 Individuals: Representing Concrete Instances](#)
 - [1.4.3 Properties: Defining Relationships](#)
- [1.5 Ontology vs Traditional Data Modeling](#)
- [1.6 Open World Assumption: A New Way of Thinking](#)
- [1.7 Reasoners: The Engine Behind Semantic Intelligence](#)
- [1.8 Why Ontology Matters in the AI Era](#)
- [1.9 What You Will Build Throughout This Book](#)
- [1.10 The Bigger Picture](#)
- [Next Chapter Preview](#)
- [Reference](#)
- [Demo Video for this Chapter](#)
- [One More Thing...](#)

1.1 Why the Pizza Tutorial Became an Industry Classis

The **pizza.owl** tutorial has become the “Hello World” of ontology engineering for several reasons.

First, pizza is universally understandable. Nearly everyone intuitively understands toppings, ingredients, vegetarian classifications, spicy foods, and regional styles. This makes it easier to focus on semantic modeling concepts rather than domain complexity.

Second, the tutorial gradually introduces increasingly sophisticated OWL concepts in a controlled environment. Instead of overwhelming learners with abstract logic theory, it teaches ontology construction incrementally:

- Classes
- Subclasses
- Individuals
- Object Properties
- Restrictions
- Reasoners
- Logical Inference
- SWRL Rules
- SPARQL Queries
- SHACL Validation

This progression mirrors how real enterprise ontology projects evolve.

Third, the tutorial demonstrates something fundamentally important:

Ontologies are not just diagrams.
They are executable semantic systems.

This is the core idea that later becomes central in EKA.

1.2 Protégé: More Than an Ontology Editor

Protégé is often introduced simply as an ontology editor, but this description dramatically understates its importance.

Protégé is better understood as a **semantic modeling platform**.

It allows users to:

- Define domain concepts
- Encode formal semantics
- Build machine-readable knowledge structures
- Execute logical reasoning
- Validate constraints
- Query semantic relationships
- Prepare foundations for knowledge graphs

In traditional enterprise architecture tools, architects often create static diagrams that describe systems visually but lack executable semantics.

For example:

- UML diagrams describe structure
- BPMN diagrams describe processes
- ER diagrams describe data
- Capability maps describe organizations

However, these artifacts usually remain disconnected documentation.

Protégé introduces a fundamentally different paradigm.

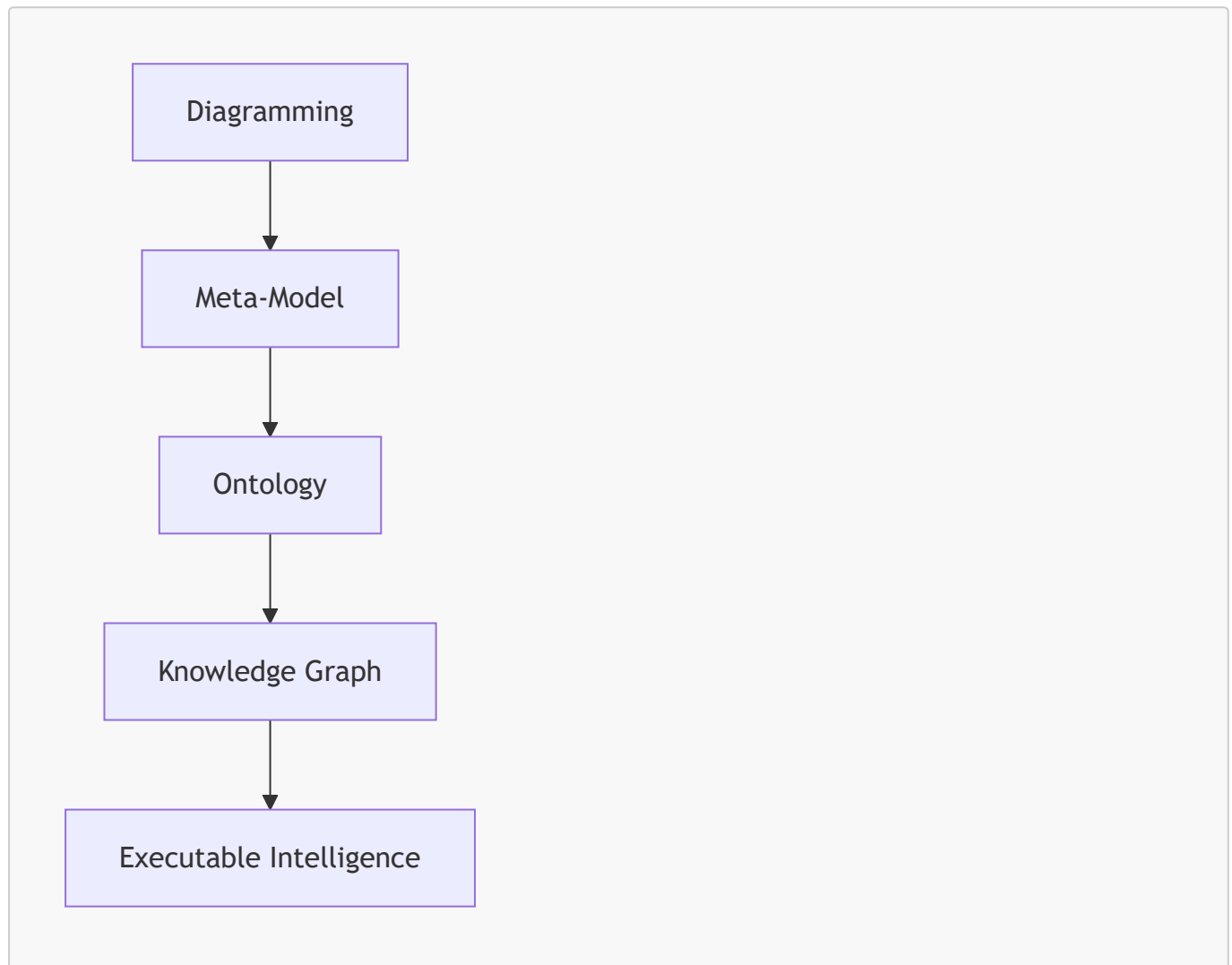
Instead of merely drawing concepts, we formally define them using **logic-based semantics**. The ontology becomes interpretable by machines, enabling automated reasoning and inference.

This transition is foundational for modern AI-native architectures.

1.3 The EKA Perspective: From Diagrams to Executable Intelligence

Within the EKA (Executable Knowledge Architecture) framework introduced on xiaoqi.com, ontology engineering occupies a critical transformation layer.

The implementation roadmap of EKA can be summarized as:



The Pizza OWL tutorial specifically addresses the "Ontology" stage.

To understanding why this matters, we must examine the limitations of traditional enterprise modeling.

Most enterprise architecture repositories contain thousands of diagrams:

- Application landscapes
- Process flows
- Integration maps
- Data models
- Organizational charts

But these assets - very valuable - are typically:

- Static
- Human-readable only
- Semantically ambiguous
- Non-executable
- Difficult to integrate

Ontology changes this situation fundamentally.

When enterprise concepts become ontological entities:

- Relationships gain formal meaning
- Constraints become machine-processable
- Inference becomes possible
- Knowledge graphs can be generated
- AI systems can consume structured semantics

In EKA terminology, ontology acts as the semantic bridge between abstract architecture and executable intelligence.

This is why learning Protégé is not merely learning another modeling tool.

IT IS learning how to formalize knowledge itself.

1.4 Understanding OWL: The Semantic Core

The tutorial centers around OWL, the Web Ontology Language.

[Web Ontology Language](#) is a W3C standard designed to represent knowledge in a machine-readable and logically rigorous form.

Unlike relational databases, OWL is not primarily designed for transaction processing.

Instead, OWL focuses on:

- Meaning
- Classification
- Inference
- Semantic relationships
- Knowledge interoperability

Michael DeBellis emphasizes that OWL ontologies are composed primarily of three core elements:

1. Classes
2. Properties
3. Individuals

These concepts form the semantic building blocks of all ontology systems.

1.4.1 Classes: Modeling Conceptual Categories

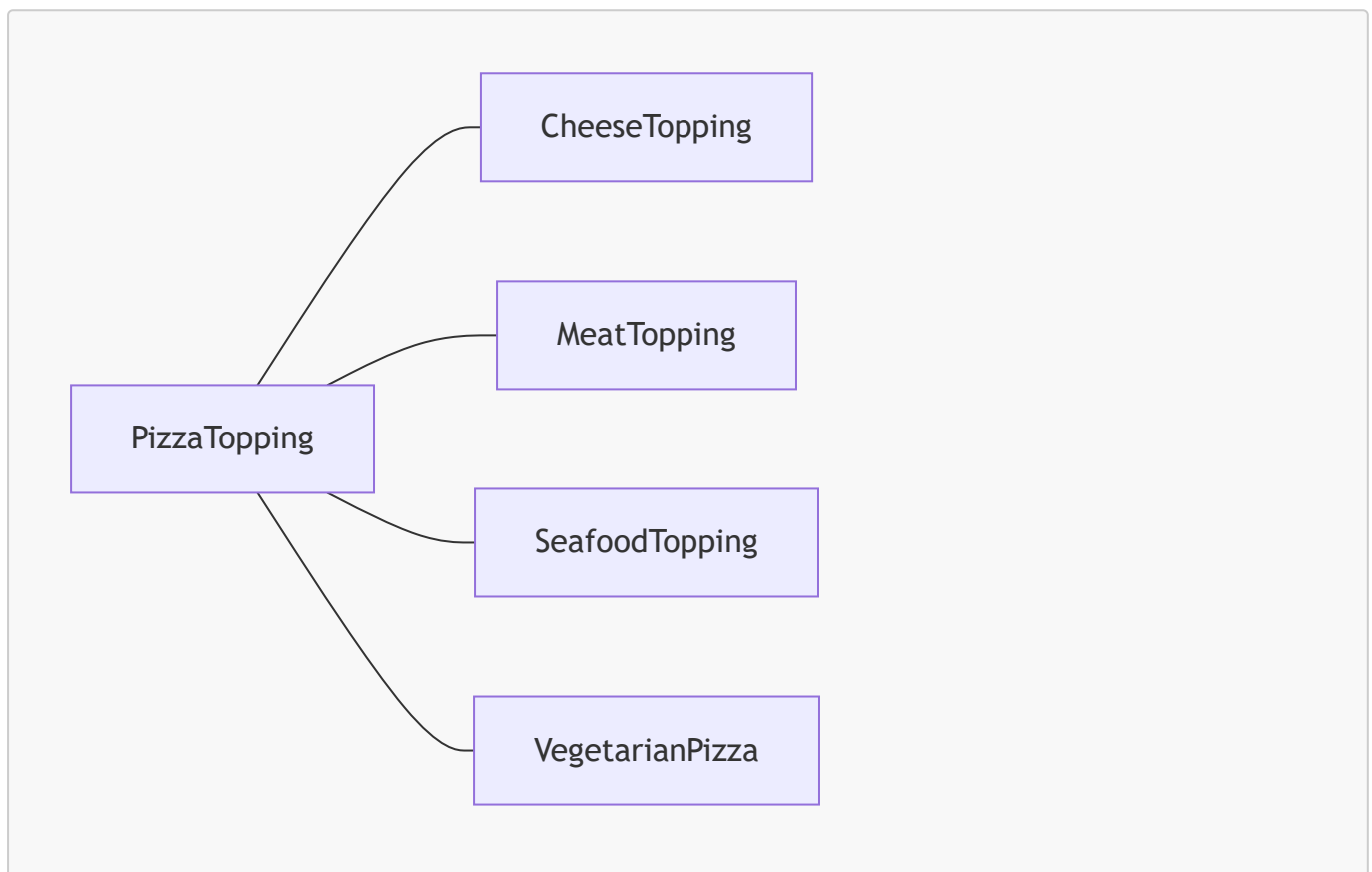
A class represents a category or type of thing.

Examples in the Pizza ontology include:

- Pizza
- PizzaTopping
- CheeseTopping
- VegetableTopping
- VegetarianPizza

Classes can be organized hierarchichally.

For example:



You may also document above graph in this way:

```
PizzaTopping
|-- CheeseTopping
|-- MeatTopping
|-- SeafoodTopping
|-- VegetableTopping
```

This resembles inheritance structure in object-oriented modeling, but OWL classes are fundamentally semantic rather than implementation-oriented.

A class in OWL describes a set of possible individuals.

This distinction becomes important later when reasoning engines classify instances automatically.

1.4.2 Individuals: Representing Concrete Instances

Individuals are actual members of classes.

For example, members of class `Pizza` or `PizzaTopping` can be:

- `MargheritaPizza`
- `MozzarellaTopping`
- `SpicyBeefPizza`

An individual belongs to one or more classes.

However, unlike traditional systems where classification is often manually assigned, OWL reasoners can infer class membership automatically base on logical definitions.

This capability is one of the major breakthroughs of ontology systems.

1.4.3 Properties: Defining Relationships

Properties define how entities relate to one another.

In OWL, there are two major property types:

1. Object Properties: These connect individuals to other individuals.

Example:

```
hasTopping
```

2. Data Properties: These connect individuals to literal value.

Example:

```
hasCalories  
hasPrice  
hasSpicinessLevel
```

Properties become extraordinarily powerful when combined restrictions and logical constraints.

1.5 Ontology vs Traditional Data Modeling

New learners often ask:

"How is ontology different from a database schema?"

This is one of the most important conceptual distinctions in semantic engineering.

Traditional databases focus on storing data efficiently.

Ontologies focus on representing meaning formally.

A relational schema might define:

```
Pizza(id, name)
ToppingI(id, name)
PizzaTopping(pizza_id, topping_id)
```

But the schema itself does not formally express semantic truths such as:

- Vegetarian pizzas cannot contain meat toppings
- Spicy pizzas contain spicy ingredients
- Certain toppings are subclasses of vegetable toppings
- Seafood toppings are disjoint from meat toppings

OWL allows these semantics to be explicitly modeled.

This is why ontologies become foundational for AI and knowledge graph systems.

1.6 Open World Assumption: A New Way of Thinking

One of the most difficult concepts for beginners is the Open World Assumption (OWA).

Traditional databases generally operate under a Closed World Assumption (CWA):

If something is not stored, it is considered false.

OWL works differently!

Under OWA:

Absence of knowledge does not imply falsehood.
It just means the knowledge that is unknown yet.

For example:

If an ontology does not specify whether a pizza contains meat, we cannot conclude that it is vegetarian.

This principle aligns much more closely with real-world knowledge representation, where information is often incomplete.

Understanding OWA is essential because it fundamentally changes how semantic systems behave compared to traditional enterprise systems.

Michael DeBellis highlights this repeatedly throughout the tutorial because many modeling errors originate from misunderstanding **open-world reasoning**.

1.7 Reasoners: The Engine Behind Semantic Intelligence

The real magic of ontology engineering appears when we introduce reasoners.

A reasoner is a logic engine that analyzes ontology axioms and derives new knowledge automatically.

For example, if we define:

- `VegetarianPizza` $\equiv$ `Pizza` AND NOT (`hasTopping` some `MeatTopping`)

and later create a pizza that only contains cheese and vegetables, the reasoner can automatically classify it as a `VegetarianPizza`.

No manual tagging is required!

This is a profound shift.

The ontology stops being passive documentation and becomes an executable semantic system.

This is precisely the transition EKA describes as moving toward executable intelligence.

1.8 Why Ontology Matters in the AI Era

Modern AI systems increasingly require structured semantic context.

Large Language Models (LLMs) are powerful, but without explicit knowledge structures they often struggle with:

- Consistency
- Explainability
- Governance
- Enterprise semantics
- Controlled reasoning

Poor structures for LLMs normally lead to the "beautiful garbage" to be generated by AI.

Ontology provides the semantic backbone that complements probabilistic AI systems.

This is why ontology engineering is re-emerging as a critical discipline in:

- Knowledge Graphs
- Semantic Search
- AI Agents
- Digital Twins
- Enterprise Knowledge Management
- Data Fabric architectures

The Pizza tutorial may appear simple, but the underlying modeling principles scale into enterprise-grade semantic systems.

1.9 What You Will Build Throughout This Book

As this eBook progresses, we will gradually build increasingly sophisticated ontology capabilities using Protégé.

The journey includes:

- Constructing class hierarchies
- Modeling object properties
- Defining logical restrictions

- Using automated reasoners
- Performing semantic queries
- Applying SWRL rules
- Validating ontologies with SHACL
- Connecting ontology concepts to knowledge graph thinking
- Positioning ontology inside the EKA execution pipeline

By the end, readers should understand not only how to use Protégé, but why ontology engineering matters strategically for modern AI-native enterprises.

1.10 The Bigger Picture

The Pizza ontology is not really about pizza.

It is about learning how to formalize knowledge.

Once you understand:

- classes,
- relationships,
- restrictions,
- inference,
- and semantic reasoning

you can model almost anything:

- enterprise architectures
- business capabilities
- manufacturing systems
- healthcare vocabularies
- supply chains
- digital twins
- AI agent memory systems
- and knowledge graphs

See https://github.com/yasenstar/ArchiMate_Ontology as one practical sample that I'm applying to use Protégé to build ontology for ArchiMate (EA Modeling language).

This is where ontology engineering evolves from academic theory into enterprise intelligence infrastructure.

The future of architecture is not static diagrams.

The future is executable semantics.

And Protégé is one of the gateways into that world.

1.11 Chapter 01 Summary

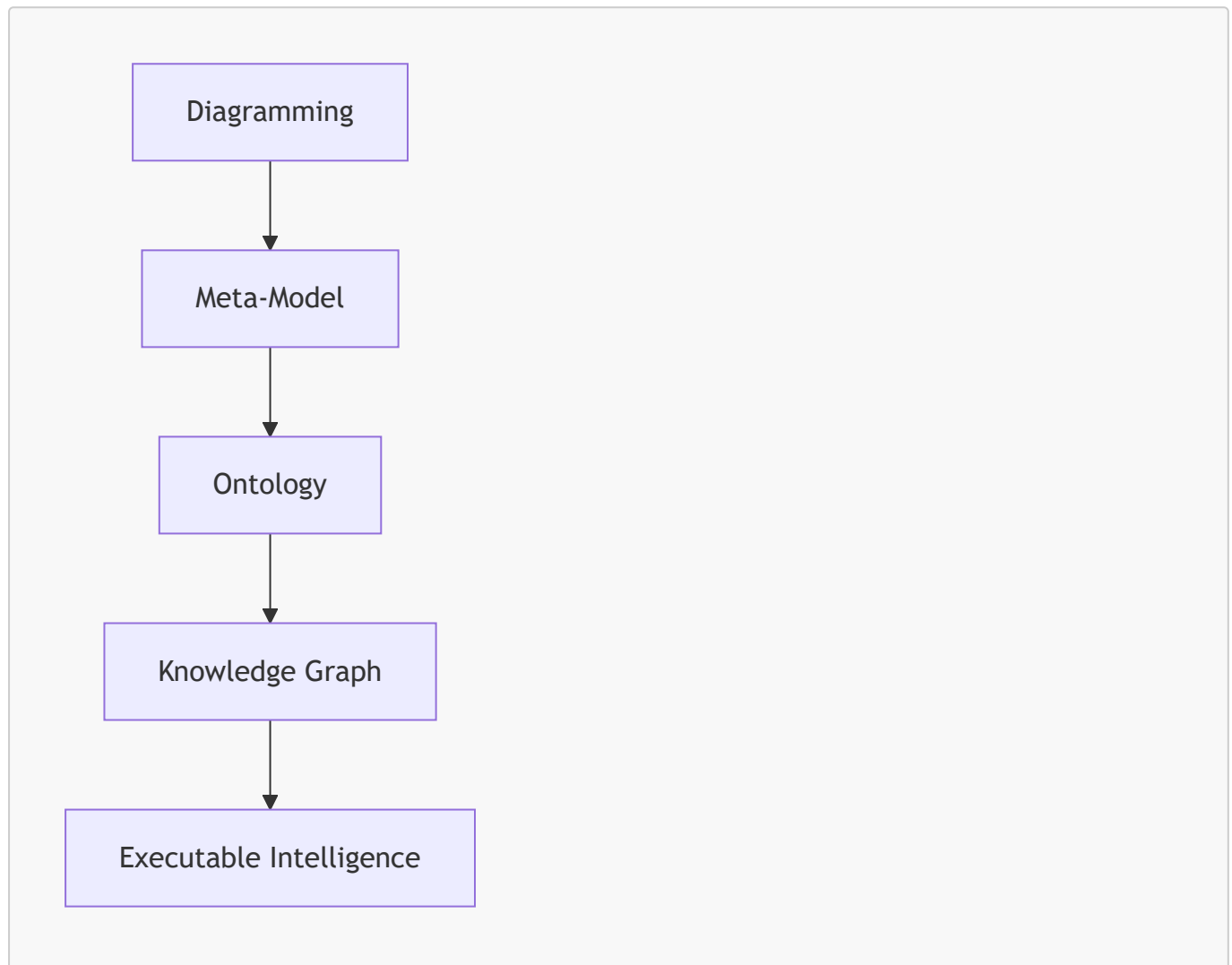
In this chapter, we established the conceptual and strategic foundations of ontology engineering using the famous [pizza.owl](#) tutorial as the entry point. Rather than treating Protégé as merely a standalone modeling

application (ontology editor), we positioned it as a semantic engineering platform capable of transforming static knowledge into executable intelligence.

We explored why the Pizza Tutorial became one of the most influential ontology learning examples in the Semantic Web community and examined how Protégé enables users to formally define concepts, relationships, restrictions, and semantic logic through OWL ontologies.

The chapter also introduced the EKA (Executable Knowledge Architecture) perspective, where ontology acts as the semantic bridge between enterprise modeling and knowledge-driven intelligence systems.

Within the EKA implementation roadmap:



the Pizza OWL tutorial specifically focuses on the ontology layer, helping learners understand how formal semantic structures are created and reasoned upon.

In addition, we introduced several core OWL concepts including:

- Classes
- Individuals
- Properties
- Open World Assumption
- Reasoners
- Semantic Inference

These concepts form the foundation for all future ontology engineering activities throughout this book.

Finally, we connected ontology engineering to the broader AI era, explaining why semantic modeling, knowledge graphs, and executable semantics are becoming increasingly important in modern enterprise architecture and intelligence systems.

Next Chapter Preview

In the next chapter, we will begin working directly inside Protégé and start constructing the foundational structure of the Pizza ontology.

Readers will learn how to:

- Create ontology projects
- Understand Protégé workspace components
- Build class hierarchies
- Create subclasses
- Define semantic categories
- Organize ontology structures properly

This marks the beginning of hands-on ontology engineering using Protégé.

The transition from theory to implementation starts there.

Reference

- Michael DeBellis, A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins
- Protégé official resources and tutorials
- Protégé Pizza Ontology learning repository: https://github.com/yasenstar/protege_pizza
- EKA Official Website: <https://xiaoqi.com->

Demo Video for this Chapter

Visit <https://youtu.be/l0PZhqmTwfM> for watching this video in YouTube.

One More Thing...

Small tip, how to type **Protégé** in Markdown with those French é?

Answer: 😊, just using your keyboard's number pad, hold **Alt** key and key in **0233** or **233**, like **Alt + 0233** or **Alt + 233**, then you see the magic. (This is called Alt Code.) Enjoy!

Last updated at: 2026/05/11

Chapter 02 - Building Your First Ontology in Protégé

After establishing the conceptual foundations of ontology engineering in Chapter 01, we now move into the practical world of ontology construction inside Protégé. This chapter marks the transition from semantic theory into hands-on ontology modeling.

The goal of this chapter is not merely to "click through a tool," but to understand how semantic structures are engineered systematically. In traditional software engineering, developers write executable code. In ontology engineering, architects build executable meaning.

This distinction is fundamental.

The Pizza ontology may appear simple on the surface, but it introduces modeling patterns used throughout enterprise semantic systems, knowledge graphs, and AI-driven architectures.

This chapter is primarily based on the ontology construction sections from Michael DeBellis' revised [Pizza Tutorial for Protégé 5.5](#).

- [2.1 From Theory to Construction](#)
- [2.2 Understanding the Protégé Workspace](#)
- [2.3 Creating a New OWL Ontology](#)
- [2.4 The Importance of Ontology Naming Conventions](#)
- [2.5 Understanding **Thing**: The Root of All Classes](#)
- [2.6 Creating the Core **Pizza** Taxonomy](#)
- [2.7 Building Class Hierarchies Systematically](#)
- [2.8 Disjoint Classes: Preventing Semantic Contradictions](#)
- [2.9 Primitive vs Defined Classes](#)
 - [2.9.1 Primitive Class](#)
 - [2.9.2 Defined Class](#)
- [2.10 Why Automated Classification Matters](#)
- [2.11 Ontology Engineering vs Diagramming](#)
- [2.12 Early Modeling Discipline](#)
- [2.13 The Emerging Semantic Architecture Mindset](#)
- [Chapter 02 Summary](#)
- [Key OWL Concepts](#)
- [Protégé Operations \(Skill\) Learned](#)
- [EKA Connection](#)
- [Knowledge Graph Perspective](#)
- [Next Chapter \(03\) Preview](#)
- [Reference](#)
- [Demo Video for this Chapter](#)

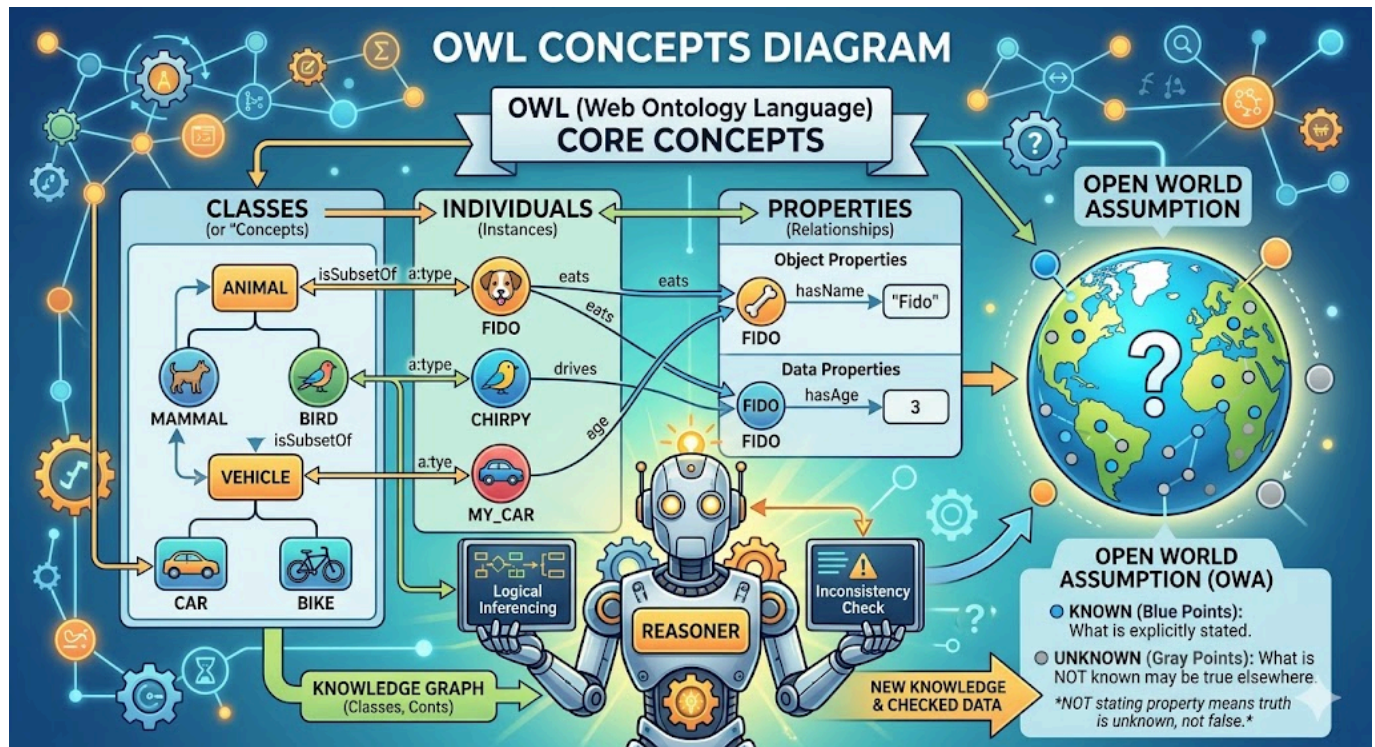
2.1 From Theory to Construction

In Chapter 01, we introduced the core OWL concepts:

- [Classes](#)

- Individuals
- Properties
- Reasoners
- Open World Assumption

An quick AI-generated picture below brings them together systematically:



Now we begin building actual semantic structures.

This is where ontology engineering starts to feel very different from traditional modeling disciplines.

In a traditional diagramming tool, creating a class hierarchy is often just a visual activity. In Protégé, however, every modeling action contributes to a machine-processable semantic structure.

This means that:

- class hierarchies have logical implications
- property definitions affect reasoning behavior
- restrictions become executable constraints, and
- inference engines can derive new knowledge automatically

As Michael DeBellis emphasizes throughout the tutorial, ontology engineering should be approached as semantic knowledge modeling rather than merely graphical editing.

2.2 Understanding the Protégé Workspace

When Protégé is launched for the first time, many new users feel overwhelmed by the interface.

This is normal.

Protégé is not designed as a lightweight diagramming application. On the contrary, it is a **semantic engineering environment**.

The workspace contains multiple perspectives for editing different ontology components:

- Classes
- Object Properties
- Data Properties
- Individuals
- Annotations
- Reasoner views
- Query views
- Rule views

One of the most important concepts for beginners to understand is that Protégé does not separate "diagramming" from "logic".

Everything visible in the interface ultimately contributes to OWL axioms underneath.

This is why ontology engineering feels closer to knowledge programming than traditional modeling.

2.3 Creating a New OWL Ontology

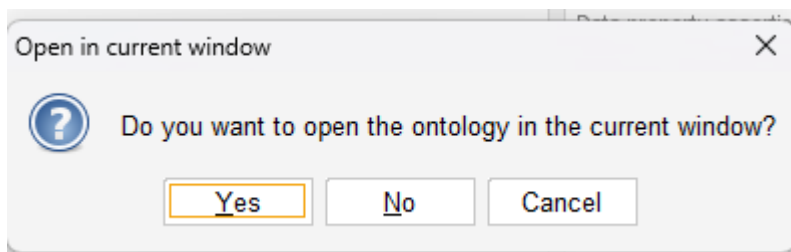
The first practical step is creating a new ontology project.

During this chapter's writing, I'm using Protégé v5.6.9.

Inside Protégé: **File** > **New...**:



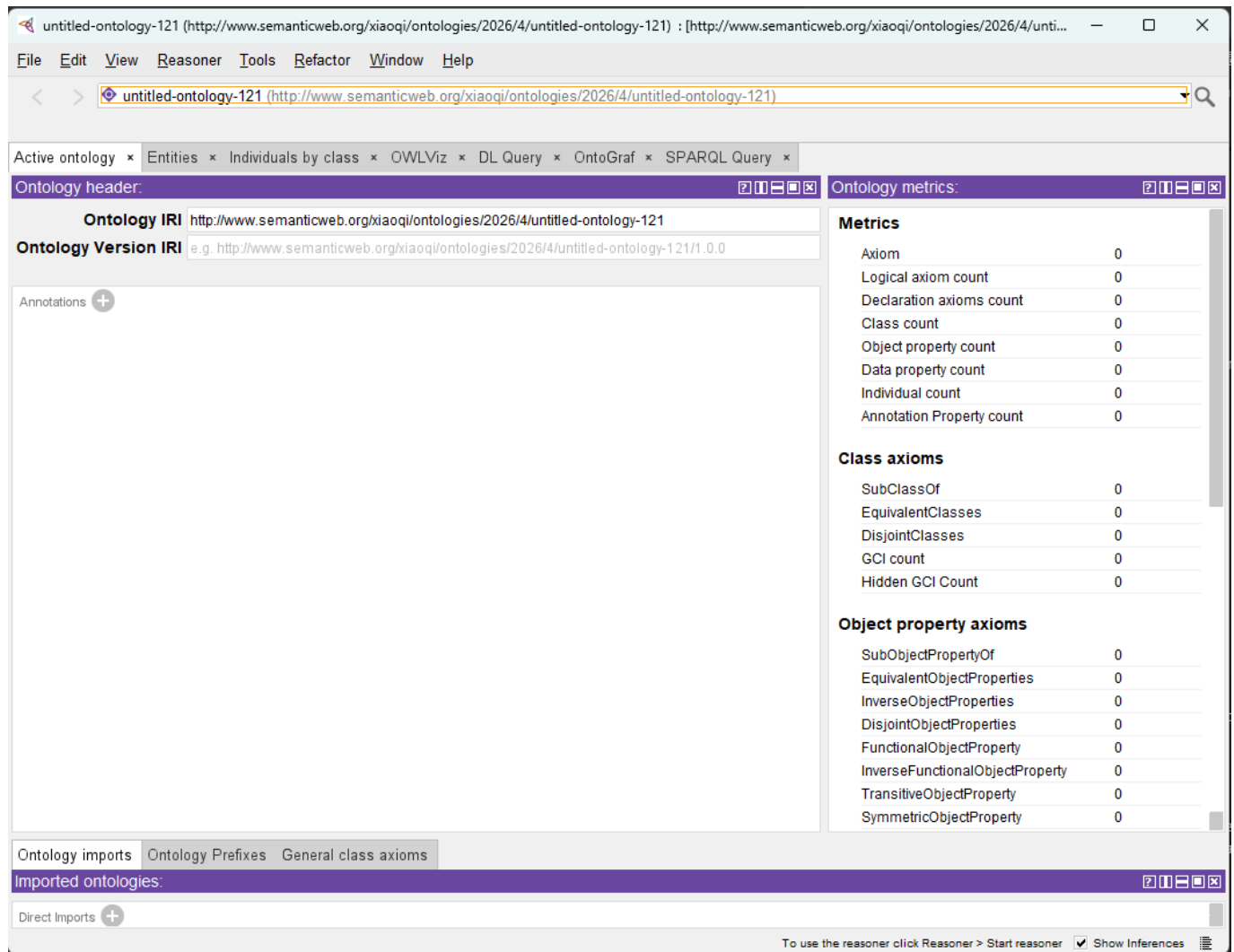
If you're having existing ontology file opened in Protégé, you will get below pop-up with question:



If you choose **Yes** (default), your current ontology file will be closed and your new ontology project will occupy current window, ensure you save any previous changes before losing them.

Or, you may choose **No** so a new window will be opened for your new ontology project, your existing opened ontology will not be impacted.

You then can see the new ontology project screen:



When you want to save the project, Protégé then asks for:

- Ontology IRI
- Document storage location
- RDF/XML or OWL serialization format

At this stage, beginners often underestimate the importance of ontology [IRIs - Internationalized Resource Identifier](#).

As defined by W3C, "Each ontology may have an ontology IRI, which is used to identify an ontology." An ontology IRI acts as the global semantic identifier for the ontology.

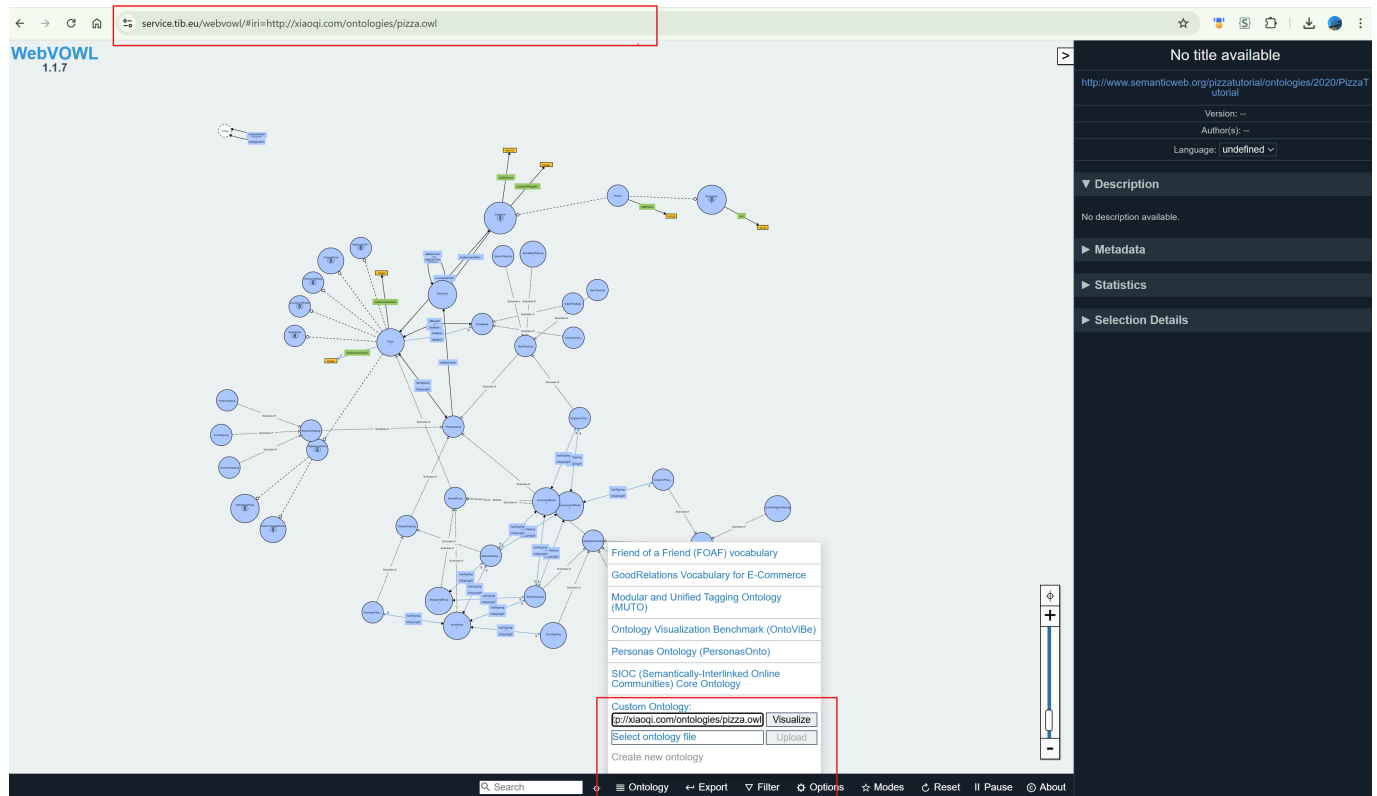
Unlike filenames or database table names, ontology IRIs are intended to be globally unique identifiers in semantic ecosystems.

For example:

```
# IRI for pizza.owl
http://xiaoqi.com/ontologies/pizza.owl

# My new ontology project IRI (local)
http://www.semanticweb.org/xiaoqi/ontologies/2026/4/untitled-ontology-121
```

Using [WebVOWL - the OWL Visualization Tool](#), you can paste above `pizza.owl` IRI then visualize it for now:



This is conceptually similar to namespaces in programming languages, but with web-scale semantic interoperability.

In enterprise environments, ontology naming strategies become critically important because ontologies may later integrate across:

- business domains
- data catalogs
- APIs
- knowledge graphs
- AI agents
- and semantic search platforms

This is one of the first points where ontology engineering begins to intersect with enterprise architecture governance.

2.4 The Importance of Ontology Naming Conventions

One of the subtle but important lessons from the Pizza tutorial is disciplined naming.

Ontology projects quickly become unmanageable if naming conventions are inconsistent.

Michael DeBellis repeatedly emphasizes semantic clarity and hierarchy organization throughout the tutorial.

Good ontology naming should satisfy several criteria, as below:

- Human-readable
- Machine-processable
- Semantically meaningful

- Consistent across domains
- Stable over time

For example in our `pizza.owl`:

```
Pizza
PizzaTopping
VegetarianPizza
SpicyPizza
```

These names are concise and semantically clear.

In enterprise ontology projects, naming conventions become even more important because ontologies may evolve into shared organizational semantic standards.

Within EKA, naming discipline is essential because ontology entities later become:

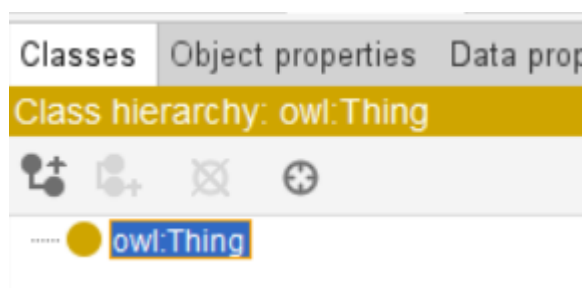
- graph nodes
- semantic APIs
- AI context objects
- digital twin entities
- and executable business semantics

Poor naming eventually creates **semantic debt** and chaos across the organization.

This is analogous to **technical debt** in software engineering.

2.5 Understanding `Thing`: The Root of All Classes

Every OWL ontology begins with a universal root class: `owl:Thing`



All classes ultimately inherit from `Thing`.

This concept initially appears trivial, but it represents an important philosophical principle in ontology engineering.

OWL assumes that all modeled entities exist within a universal semantic space.

For example:

```
Thing
|-- Pizza
```

```
| -- PizzaTopping  
| -- PizzaBase  
| -- Spiciness
```

Unlike relational databases where tables are independent structures, OWL organizes concepts into a globally connected semantic universe.

This is one reason ontology systems integrate naturally with knowledge graphs.

2.6 Creating the Core **Pizza** Taxonomy

The next step involves building the foundational class hierarchy.

From Wikipedia, **Taxonomy** is a practice and science concerned with classification or categorization.

The Pizza ontology begins with several top-level conceptual categories:

```
Pizza  
PizzaTopping  
PizzaBase  
Spiciness
```

These represent the primary semantic domains within the ontology.

At this stage, ontology engineering resembles taxonomy construction.

However, ontologies extend far beyond simple taxonomies.

An ontology additionally defines:

- semantics
- constraints
- properties
- logical restrictions
- inference behavior
- and reasoning rules

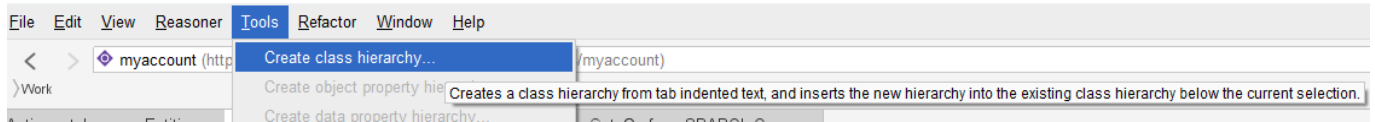
This distinction is critical.

Many enterprise "ontologies" are actually only taxonomies because they lack formal semantics.

2.7 Building Class Hierarchies Systematically

One of the most powerful productivity features demonstrated in the Pizza tutorial is the "Create Class Hierarchy" wizard.

In Protégé, from menu [Tools] > [Create class hierarchy...]



Instead of manually creating classes one by one, Protégé allow ontology engineers to rapidly construct semantic hierarchies using indentation-based structure.

For example:

```
PizzaTopping
  CheeseTopping
  MeatTopping
  SeafoodTopping
  VegetableTopping
```

Tip: you may create those structure in one TXT file upfront, then paste into the wizard.

This approach provides several advantages:

- Faster ontology construction
- Better structural consistency
- Easier semantic organization
- Able to see **disjoint** in one go (see [2.8](#) below)
- Reduced modeling errors

In enterprise-scale ontology projects containing thousands of concepts, hierarchical discipline becomes essential.

Without strong hierarchy governance:

- semantic duplication emerges
- inheritance becomes inconsistent
- and reasoning performance degrades

Ontology engineering therefore requires architectural thinking, not merely modeling skills.

2.8 Disjoint Classes: Preventing Semantic Contradictions

An extremely important concept introduced during hierarchy creation is **class disjointness**.

For example:

```
MeatTopping
VegetableTopping
Seafoodtopping
```

These categories should logically be mutually exclusive.

A topping cannot simultaneously be both:

- MeatTopping
- and VegetableTopping

Protégé allows classes to be declared as disjoint.

This enables reasoners to detect contradictions automatically.

For example, if an individual were incorrectly classified as both:

```
HamTopping
AND
VegetableTopping
```

Since **HamTopping** is one type of **MeatTopping**, the reasoner would identify an inconsistency.

This capability becomes enormously important in enterprise environments.

The Disjointness supports:

- data quality enforcement
- semantic governance
- AI validation
- and knowledge consistency checking

Within EKA, this is one of the first places where architecture becomes executable.

The ontology no longer merely describes the domain, it actively validates semantic correctness.

2.9 Primitive vs Defined Classes

One of the most intellectually important concepts in OWL modeling is the distinction between:

- Primitive Classes
- Defined Classes

2.9.1 Primitive Class

A primitive class only specifies necessary conditions.

For example:

```
Pizza
```

may simply be declared as a subclass of:

```
Food
```

without fully defining what constitutes a pizza.

2.9.2 Defined Class

A defined class, however, specifies both:

- necessary conditions
- and sufficient conditions

You may find similar terms in Sets Theory, correct, they're inherited from that theory indeed.

For example:

```
VegetarianPizza  
  ≡ Pizza  
  AND NOT (hasTopping some MeatTopping)
```

This means the reasoner can automatically infer membership.

If a pizza satisfies the logical conditions, it becomes classified automatically.

This is one of the defining characteristics of semantic systems.

Traditional systems rely heavily on manual categorization, while, ontology systems support computational classification.

2.10 Why Automated Classification Matters

This concept may initially seem academic, but it becomes transformative at enterprise scale.

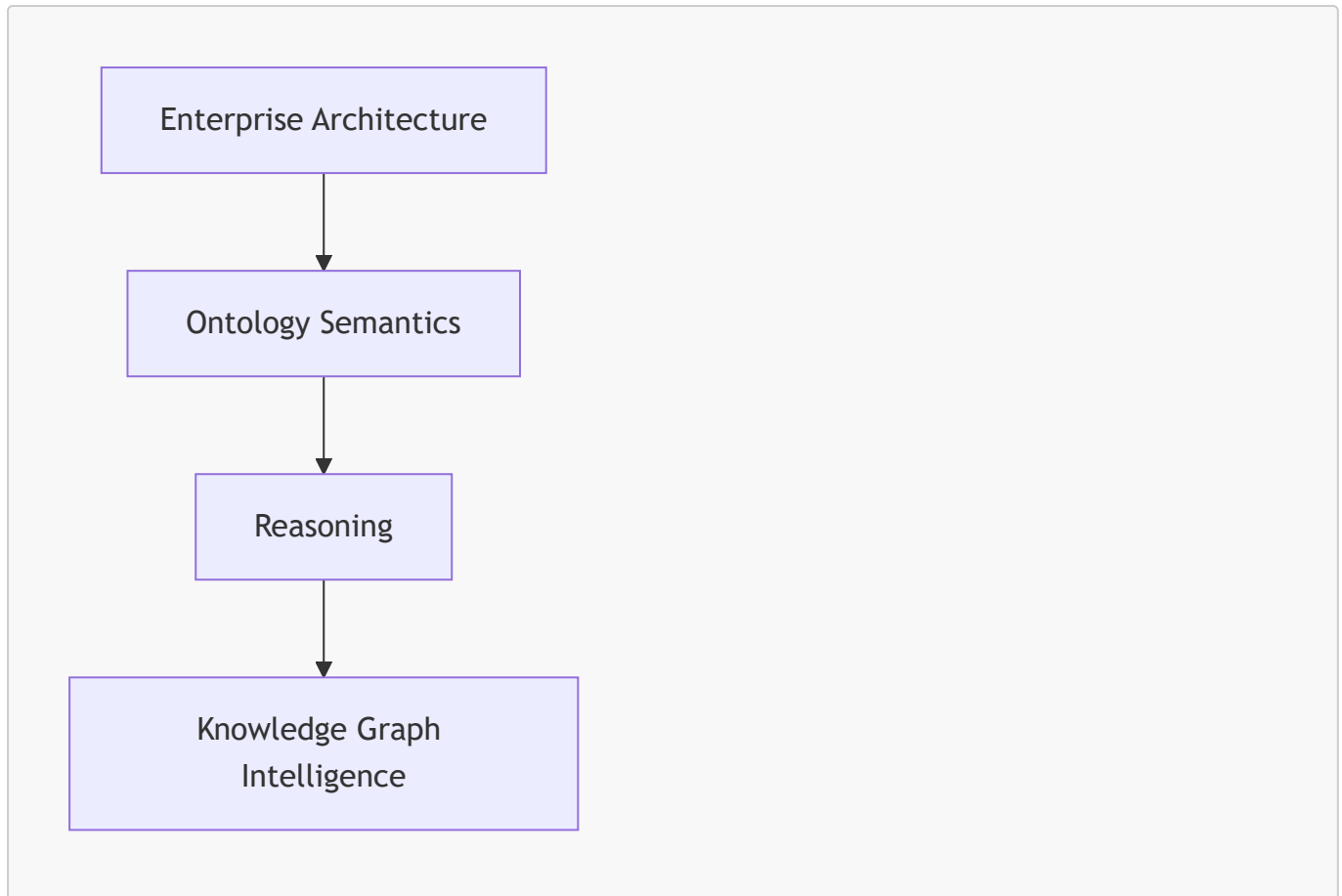
Consider a large organization containing:

- thousands of applications,
- millions of data entities,
- thousands of APIs,
- and hundrds (or possible thousands as well) of business capabilities

Manually classifying all semantic relationships become impossible.

Ontology reasoning introduces automation into semantic governance.

Within EKA, automated classification enables:



This is one of the major strategic advantages of semantic architecture over static repositories.

2.11 Ontology Engineering vs Diagramming

A common beginner's mistake is treating Protégé like Visio or UML modeling software.

This is fundamentally incorrect.

In diagramming tools:

- relationships are often visual only
- semantics remain informal
- and diagrams rarely execute

While in ontology engineering:

- semantics are formalized
- relationships are machine-readable
- and logic becomes executable

This difference is profound.

Ontology engineering sits much closer to software engineering and knowledge programming than to traditional architecture diagramming.

Protégé therefore represents a semantic compiler environment rather than merely a drawing tool. (In fact, you don't have actual "drawing" activities in Protégé.)

This distinction becomes central within EKA because the ultimate goal is not documentation.

The goal is executable knowledge.

2.12 Early Modeling Discipline

One of the most valuable lessons from the Pizza tutorial is modeling discipline.

Good ontology engineering requires:

- semantic clarity,
- naming consistency,
- hierarchy governance,
- logical rigor,
- and iterative refinement

New ontolgoey engineers often attempt to model everything immediately.

Experienced ontology architects instead:

1. establish clean top-level structures,
2. define semantic boundaries,
3. create reusable patterns,
4. and evolve the ontology incrementally.

This mirrors good enterprise architecture practices.

Ontology engineering is therefore both:

- a technical discipline,
- and an architectural discipline.

2.13 The Emerging Semantic Architecture Mindset

At this stage in the learning journey, you should begin recongnizing a major conceptual transition.

Traditional enterprise systems are largely:

Data-centric

Ontology systems are

Meaning-centric

This shift has enormous implications for AI systems.

Modern intelligent systems increasingly require:

- semantic interoperability,
- contextual understanding,
- knowledge reasoning,

- and machine-readable meaning

Ontology engineering provides the formal semantic layer necessary for this evolution.

This is why ontology, knowledge graphs, and semantic AI are rapidly converging across modern enterprise architecture.

Chapter 02 Summary

In this chapter, we began constructing the foundational structure of the Pizza ontology inside Protégé.

We have explored together:

- the Protégé workspace,
- ontology project initialization,
- ontology IRIs,
- semantic naming conversions,
- class hierarchy creation,
- disjoint classes,
- primitive vs. defined classes,
- and automated classification concepts.

More importantly, we examined how ontology engineering fundamentally differs from traditional diagramming approaches.

Protégé was introduced not simply as a modeling tool, but as a semantic engineering environment capable of creating executable knowledge structures.

The chapter also reinforced the EKA perspective that ontology serves as the semantic transformation layer between enterprise architecture and executable intelligence systems.

Kew OWL Concepts

Concept	Description
owl:Thing	The universal superclass from which all OWL classes inherit.
Taxonomy	A hierarchical classification structure organizing concepts into parent-child relationships.
Ontology	A semantic model that includes taxonomy, properties, restrictions, semantics, and reasoning logic.
Disjoint Classes	Classes declared mutually exclusive to prevent semantic contradictions.
Primitive Classes	Classes containing necessary conditions only.
Defined Classes	Classes containing both necessary and sufficient conditions, enabling automated classification.

Protégé Operations (Skill) Learned

During this chapter, you learned how to:

- Create a new ontology
- Configure ontology IRIs
- Navigate the Protégé interface
- Create top-level classes
- Build class hierarchies
- Use the Create Class Hierarchy wizard
- Define semantic taxonomies
- Apply disjointness
- Organize ontology structures systematically

EKA Connection

Within EKA, this chapter represents the transition from static architecture modeling toward executable semantic structure.

Traditional enterprise repositories often contain disconnected diagrams.

Ontology introduces:

- formal semantics,
- machine readability,
- automated reasoning,
- and semantic consistency validation.

The class hierarchies created in this chapter are not merely documentation artifacts.

They are the foundational semantic structures that later evolve into:

- enterprise knowledge graphs,
- AI semantic layers,
- digital twins,
- and executable architectural intelligence

Knowledge Graph Perspective

Knowledge graphs require semantically meaningful entities and relationships.

The ontology structures created in this chapter provide the semantic backbone necessary for graph intelligence.

For example:

```
Pizza
  ↓ hasTopping
CheeseTopping
```

can later become graph nodes (entities) and graph edges (relationships) enriched within formal semantics.

Ontology therefore as the semantic schema layer for knowledge graphs.

Without ontology:

- graph semantics become ambiguous,
- relationships lose formal meaning,
- and reasoning capability becomes limited.

Next Chapter (03) Preview

In the next chapter, we will install Protégé and become familiar with the ontology engineering workspace used throughout the Pizza OWL tutorial series.

Readers will learn how to:

- install Protégé properly,
- navigate the Protégé interface,
- understand the major workspace areas,
- explore ontology editing perspectives,
- and begin developing the mindset of semantic engineering.

This step establishes the operational foundation for all future ontology modeling activities in the book.

Protégé is not simply a modeling application.

It is the primary semantic engineering environment where ontology structures are transformed into machine-readable knowledge models. Within the EKA framework, this represents the beginning of hands-on ontology engineering practice and the transition from conceptual understanding into executable semantic modeling.

Reference

- Protégé Pizza Repository: https://github.com/yasenstar/protege_pizza
- WebVOWL project in GitHub: <https://github.com/VisualDataWeb/WebVOWL>
- EKA Official Website: <https://xiaoqi.com>

Demo Video for this Chapter

YouTube Demo Video - Chapter 02: https://youtu.be/eWx9_zJkiUY

Last updated: 2026/05/15

Chapter 03 - Installing Protégé and Understanding the Ontology Engineering Workspace

In the previous chapters, we explored the conceptual foundations of ontology engineering and introduced the semantic thinking behind OWL and the Pizza ontology. We also discussed how ontology fits within the broader EKA (Executable Knowledge Architecture) frameworks as the critical semantic layer between meta-modeling and knowledge graph intelligence.

This chapter now moves into the practical environment where ontology engineering actually happens:

Protégé

Before building ontologies, you must first understand the ontology engineering workspace itself. Just as software developers require familiarity with their IDEs, ontology engineers must understand the tools, perspectives, editors, and semantic workflow provides by Protégé

This chapter focuses on two major objectives:

1. Installing Protégé properly
2. Understanding the Protégé user interface and semantic engineering environment

Although installation may initially appear straightforward, the deeper goal of this chapter is much more important:

Understanding Protégé not simply as software, but as a semantic engineering platform.

This chapter is aligned with the hands-on installation and UI walkthrough demonstrated in the video tutorial and builds upon the foundational theory introduced in Michael DeBellis' Pizza OWL tutorial.

- [3.1 Why Protégé Matters in Ontology Engineering](#)
- [3.2 Installing Protégé](#)
- [3.3 Protégé as an Ontology IDE](#)
- [3.4 Understanding the Protégé Workspace](#)
- [3.5 The Class Tab](#)
- [3.6 Understanding the Entity Tree](#)
- [3.7 Ontology Editing vs. Diagram Drawing](#)
- [3.8 OWL Serialization and Ontology Files](#)
- [3.9 Plugins and Extensibility](#)
- [3.10 The Importance of Semantic Workspace Familiarity](#)
- [3.11 Protégé Within the EKA Vision](#)
- [3.12 Developing the Ontology Engineering Mindset](#)
- [Chapter \(03\) Summary](#)
- [Key OWL Concepts](#)
- [Protégé Operations Learned](#)
- [EKA Connection](#)
- [Knowledge Graph Perspective](#)
- [Next Chapter Preview](#)

- [Extended Reading - Bonus](#)
 - [Fix the Unable to check for plugins Problem](#)
- [Reference](#)
- [Demo Video for this Chapter](#)

3.1 Why Protégé Matters in Ontology Engineering

Protégé is one of the most widely used ontology engineering platforms - far more than an editor - in the Semantic Web and Knowledge Graph ecosystem.

Originally developed at Stanford University, Protégé has become a foundational tool for:

- OWL ontology modeling
- Semantic Web development
- Knowledge graph engineering
- AI semantic modeling
- Enterprise ontology architecture
- Biomedical ontology research
- Linked data systems

Unlike traditional modeling tools, Protégé is designed specifically for semantic engineering.

This distinction is extremely important.

Most architecture or diagramming tools focus primarily on visualization:

- UML tools visualize structure
- BPMN tools visualize process
- ERD tools visualize data relationships

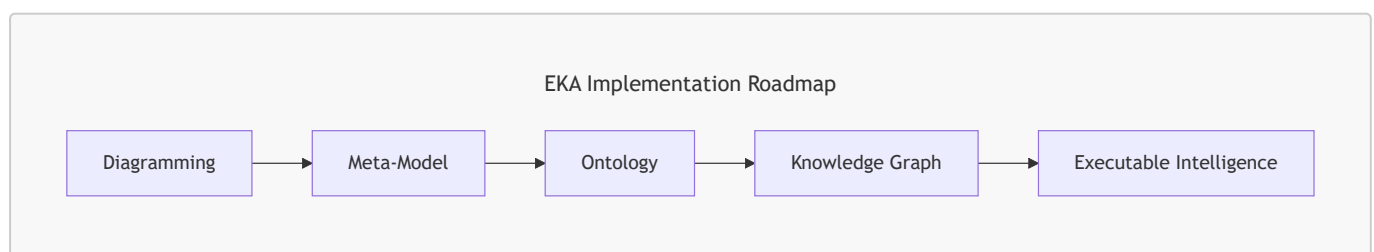
Protégé, however, focuses on:

Formal semantic meaning

Everything created inside Protégé ultimately contributes to executable semantic structures.

This is why Protégé plays such an important role with EKA.

In the EKA implementations roadmap:



Protégé becomes the primary engineering environment for the "Ontology" stage.

It is where semantic structures are formally defined before later evolving into knowledge graphs and executable intelligence systems.

3.2 Installing Protégé

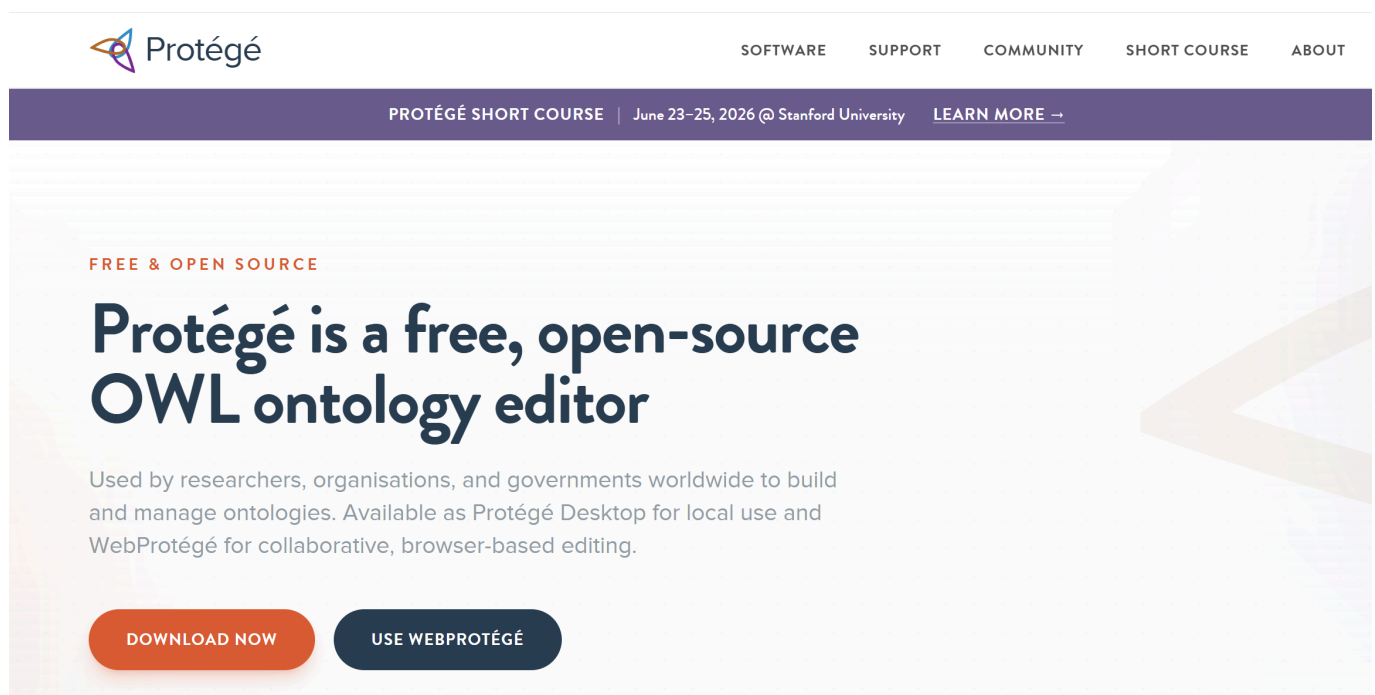
Protégé is distributed as a desktop application and supports multiple operating systems including:

- Windows
- macOS
- Linux

The official Protégé website provides downloadable installation packages:

<https://protege.stanford.edu>

You may choose **Use WebProtégé** at <https://protege.stanford.edu/software#web-protege> to experience its features before installing desktop.



The tutorial series uses Protégé 5.x, which supports modern OWL 2 ontology modeling capabilities.

Installation is generally straightforward:

1. Download the appropriate installer
2. Install Java if required
3. Launch Protégé
4. Configure workspace preferences if necessary

Although the installation process itself is relatively simple, you should understand an important architectural point:

[!Note] Protégé is not a lightweight note-taking or diagramming tool.

It is a semantic reasoning environment.

As ontology projects grow larger, Protégé becomes capable of managing:

- thousands of classes,
- semantic restrictions,
- logical inference,
- reasoning engines,
- ontology plugins,
- and graph-scale semantic models.

This is why understanding the workspace properly is essential from the beginning.

3.3 Protégé as an Ontology IDE

One of the most useful way to understand Protégé is to think of it as an IDE (Integrated Development Environment) for ontology engineering.

Software developers use environments such as:

- Visual Studio Code
- IntelliJ IDEA
- Eclipse IDE

to develop executable software.

Ontology engineers use Protégé to develop executable semantics.

This comparison helps explain why Protégé contains:

- editors,
- views,
- semantic structures,
- validation tools,
- plugins,
- reasoning engines,
- and ontology management capabilities.

Protégé is therefore not merely an editor.

It is a semantic engineering workspace.

Within EKA, Protégé can be viewed as:

An IDE for Executable Knowledge

This framing is extremely important because it changes how learners perceive ontology engineering itself.

Ontology engineering is not documentation work.

It is semantic system engineering.

3.4 Understanding the Protégé Workspace

When Protégé launches for the first time, the interface may appear complex to new users.

This is normal because Protégé exposes multiple semantic engineering perspectives simultaneously.

The workspace is designed around ontology components rather than visual diagrams.

Major areas of the interface include:

- Class hierarchy panel
- Entity editors
- Annotation views
- Object property tabs
- Data property tabs
- Individual management views
- Reasoner controls
- Ontology metadata panels

Unlike ordinary modeling tools, Protégé is organized around semantic structures.

This reflects a fundamental principle of ontology engineering:

Meaning comes before visualization

If you're familiar ArchiMate and used to Archi (the ArchiMate modeling tool), the difference of their philosophy in modeling is significant:

- Archi (ArchiMate): Diagramming First, then Relationships Second
- Protégé (Ontology): Relationships First, then Visualization Second

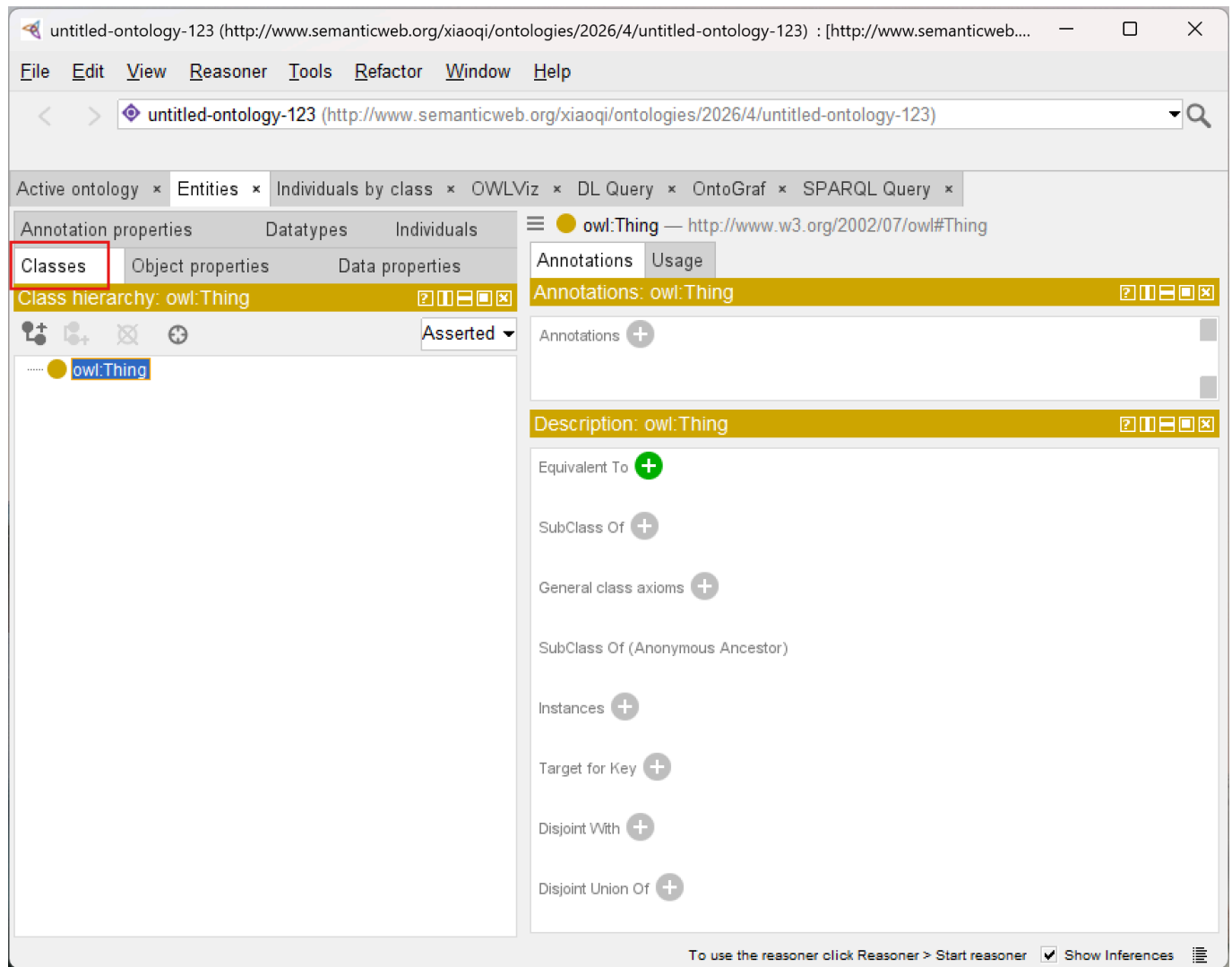
Protégé prioritizes semantic precision over graphical appearance. (see more in [3.7](#))

This may initially feel unfamiliar to users coming from traditional enterprise architecture tools.

However, this semantic-first design is one reason ontology systems scale effectively into AI and knowledge graph ecosystems.

3.5 The Class Tab

One of the first and most important areas you will encounter is the **Classes** tab.



This is where ontology taxonomies are constructed and managed.

Inside the Classes tab, users can:

- Create classes,
- organize hierarchies,
- define subclasses,
- add annotations,
- and later define logical restrictions.

At the top of the hierarchy sits **owl:Thing** which represents the universal root class in OWL.

All ontology classes ultimately inherit from **Thing**.

This is one of the first places where you begin experiencing semantic hierarchy construction directly.

Even simple class creation contributes to formal OWL semantics underneath.

3.6 Understanding the Entity Tree

The entity tree displayed inside Protégé is more than a visual navigation structure.

It represents semantic inheritance relationships.

For example:

```
PizzaTopping
|-- CheeseTopping
|-- MeatTopping
|-- VegetableTopping
```

is not simply a visual grouping.

It formally means: $\text{CheeseTopping} \subseteq \text{PizzaTopping}$.

This distinction is essential.

Protégé is continuously generating semantic axioms behind the interface.

This is one reason ontology engineering differs fundamentally from ordinary taxonomy management.

3.7 Ontology Editing vs. Diagram Drawing

Many new users initially expect Protégé to behave like a graphical modeling application such as:

- Microsoft Visio
- Sparx Systems Enterprise Architect
- Lucidchart
- Archi Tool
- draw.io, etc

However, Protégé operates differently.

Traditional modeling tools often prioritize:

- layout,
- shapes,
- visual connectors,
- and presentations.

Protégé prioritizes:

- semantic logic,
- ontology structures,
- formal meaning,
- and reasoning consistency.

This distinction becomes one of the most important mindset transitions in ontology engineering.

The ontology itself is the executable artifact.

Visualization is secondary.

Within EKA, this reflects the shift from:

Static Architecture Documentation

to:

Executable Semantic Architecture

3.8 OWL Serialization and Ontology Files

Another important concept introduced during installation and project creation is **ontology serialization**.

Protégé stores ontologies using standardized semantic formats such as:

- RDF/XML
- Turtle
- OWL/XML

These formats allow ontologies to become portable semantic assets.

Unlike proprietary diagram files, OWL ontologies can later integrate into:

- semantic APIs,
- graph databases,
- Linked Data systems,
- AI semantic layers,
- and enterprise knowledge graphs.

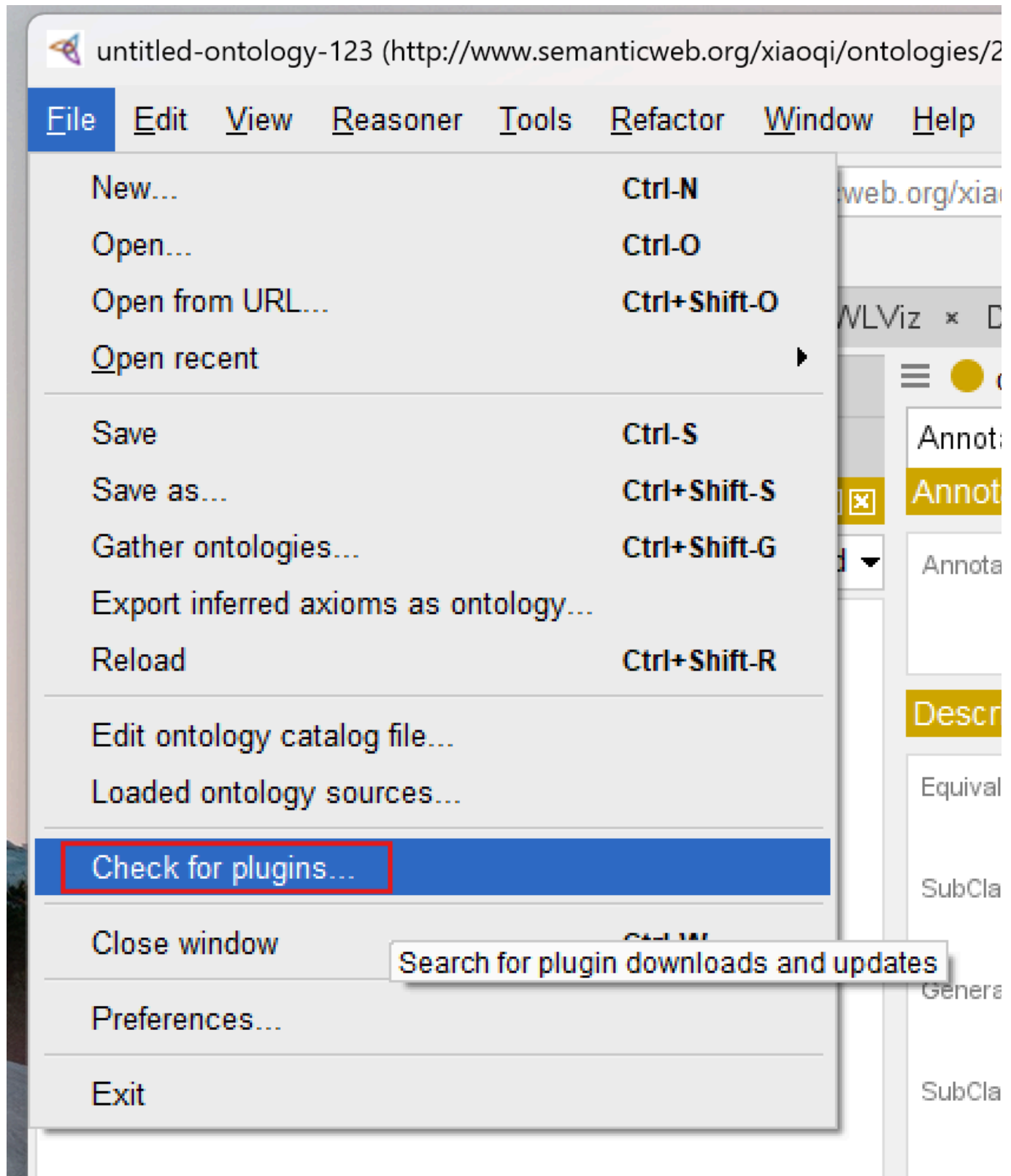
This portability is one reason ontology engineering has become increasingly important in modern AI ecosystems.

Semantic knowledge becomes reusable infrastructure.

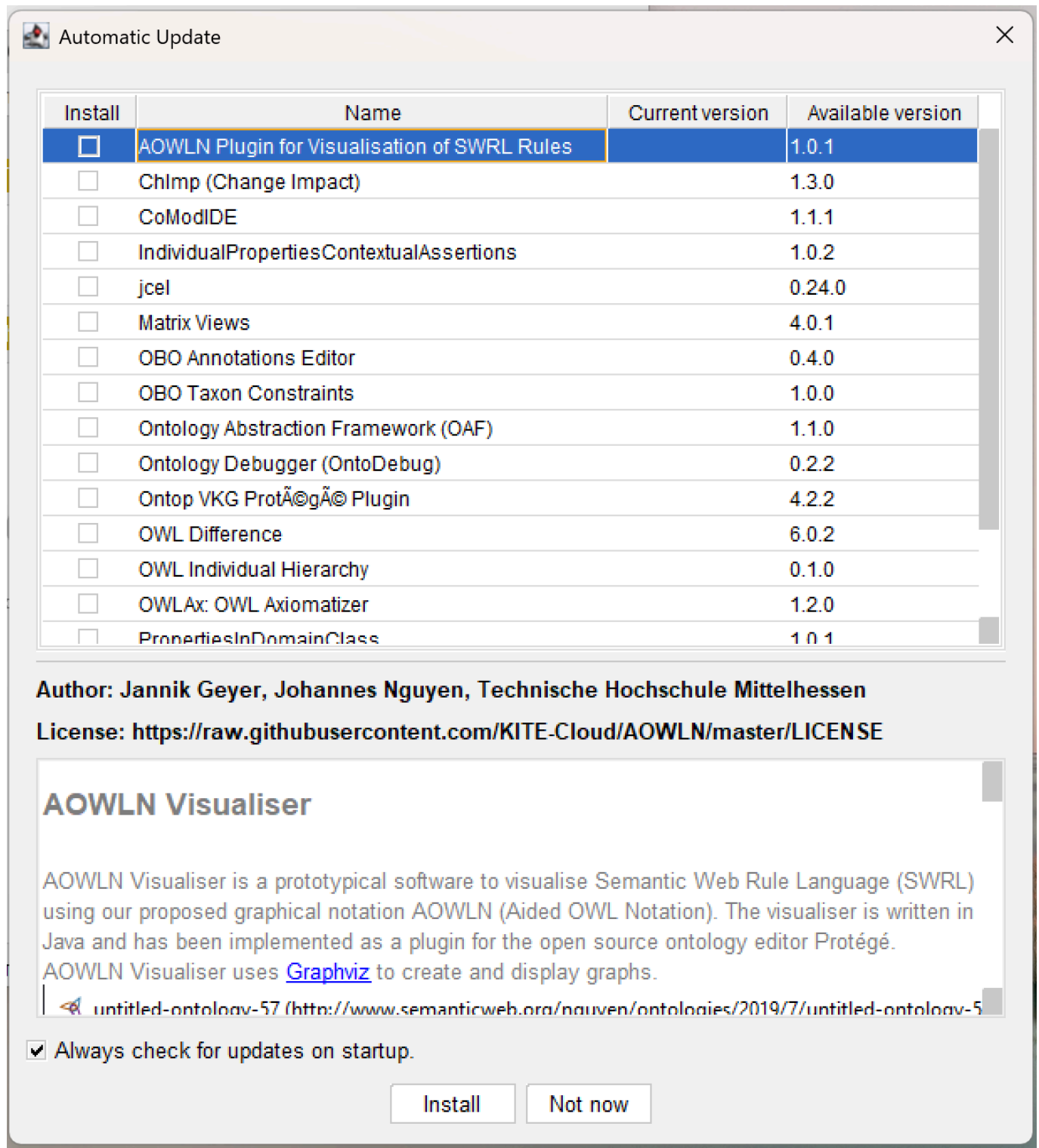
3.9 Plugins and Extensibility

Protégé also supports a plugin ecosystem.

From menu **File > Check for plugin...**, you can open the plugins window:



The plugin list will be shown and you can check the version and detail information:



This is important because ontology engineering often evolves beyond simple class modeling.

Plugins may provide capabilities such as:

- reasoning engines,
- graph visualization,
- query execution,
- ontology validation,
- semantic analysis,
- and knowledge graph integration.

As you progress further into ontology engineering, Protégé gradually transforms from a simple editor into a semantic engineering platform.

This extensibility aligns closely with EKA thinking, where ontology eventually becomes part of larger executable intelligence pipelines.

If you see the error `Unable to chekc for plugins`, check the [tip](#) in the end of this chapter - Extended Reading.

3.10 The Importance of Semantic Workspace Familiarity

One of the most underestimated aspects of ontology engineering is **workspace fluency**.

Beginners often focus entirely on OWL theory while neglecting tool mastery.

However, productive ontology engineering requires both:

- semantic understanding,
- and engineering workflow familiarity.

Experienced ontology engineers develop fluency in:

- navigating ontology structures,
- managing semantic hierarchies,
- organizing entities,
- interpreting inferred structures,
- and maintaining semantic consistency.

This is very similar to software development.

Knowing programming theory alone is insufficient without development environment proficiency.

Protégé therefore becomes part of the ontology engineer's cognitive workflow.

3.11 Protégé Within the EKA Vision

Within EKA, Protégé occupies a strategically important role.

Traditional enterprise architecture often produces:

- static diagrams,
- disconnected repositories,
- and non-executable documentations.

Protégé enables the transition into:

Machine-readable semantics

This is a foundational shift.

The ontology models built inside Protégé later become candidates for:

- semantics APIs,
- knowledge graphs,
- AI context layers,
- digital twins,
- and executable enterprise intelligence.

This is why ontology engineering matters far beyond academic Semantic Web research.

Protégé is not just a modeling application.

It is part of the infrastructure stack for intelligent architecture systems.

3.12 Developing the Ontology Engineering Mindset

At this stage, you should begin shifting from:

Diagram Thinking

toward:

Semantic Thinking

This transition is one of the MOST IMPORTANT conceptual evolutions in the entire Pizza tutorial journey.

Ontology engineering asks fundamentally different questions:

Instead of:

"What should this diagram look like?"

ontology engineering asks:

"What does this concept mean semantically?"

This semantic-first mindset later becomes essential for:

- knowledge graphs,
- semantic AI,
- enterprise reasoning,
- and executable intelligence systems.

Chapter (03) Summary

In this chapter, we installed Protégé and explored the ontology engineering workspace for the first time.

We examined:

- Protégé installation,
- the semantic engineering environment,
- ontology editing perspectives,
- class hierarchy navigation,
- OWL serialization formats,
- plugin extensibility,
- and the conceptual difference between ontology engineering and traditional diagramming.

Most importantly, we introduced Protégé as an ontology IDE and semantic engineering platform rather than merely a modeling application.

This chapter also reinforced Protégé's role within the EKA framework as the primary environment for transforming conceptual architecture into executable semantic structures.

Key OWL Concepts

Concept	Description
Ontology Editor	A software environment used to create and manage semantic ontologies.
OWL Serialization	The standarized storage format used to represent OWL ontologies.
Semantic Hierarchy	A class structure representing format inheritance relationships.
Ontology Workspace	The engineering environment used for semantic modeling and ontology management.
Semantic Engineering	The discipline of formally modeling machine-readable meaning.

Protégé Operations Learned

During this chapter, you learned how to:

- Install Protégé
- Launch the Protégé environment
- Understand the workspace layout
- Navigate ontology panels
- Explore class hierarchies
- Understand ontology file structures
- Recognize Protégé perspectives and editors

EKA Connection

Within EKA, Protégé represents the primary semantic engineering environment for ontology construction.

It enables the transition from:

Meta-Model Thinking

to:

Executable Semantic Modeling

The workspace familiarity developed in this chapter forms the operational foundation for all future ontology engineering activities inside the EKA execution pipeline.

Knowledge Graph Perspective

Protégé itself is not a graph database.

However, the semantic structures created within Protégé form the foundational layer that later enables knowledge graph generation.

The ontology classes, hierarchies, and semantic definitions managed inside Protégé eventually become graph-ready semantic assets.

Ontology engineering therefore acts as a precursor to enterprise knowledge graph engineering.

Next Chapter Preview

In the next chapter, we will begin building the Pizza ontology from the ground up inside Protégé.

You will learn how to create classes, establish subclass hierarchies, and annotate your ontology with meaningful descriptions.

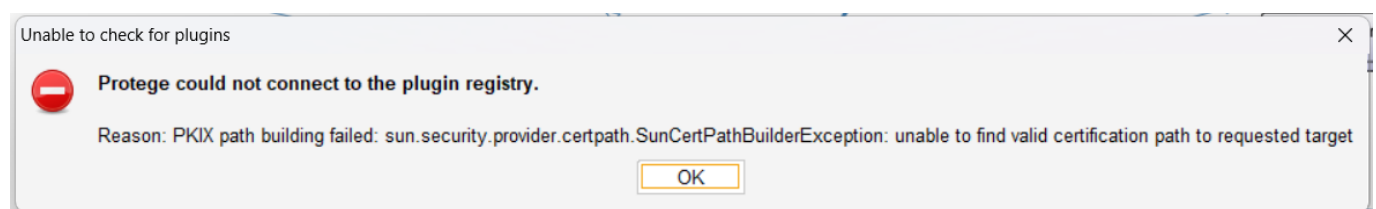
This chapter marks the first step in transforming your understanding of semantic structures into hands-on ontology modeling.

By the end, you will gain practical experience in organizing conceptual categories, preparing the foundation for more advanced semantic modeling in subsequent chapters.

Extended Reading - *Bonus*

Fix the **Unable to check for plugins** Problem

If you want to check plugins, but get below popup error:



No worry!

This error message, specifically the `SunCertPathBuilderException`, usually indicates that the Java environment Protégé is running on doesn't trust the SSL certificate of the plugin registry.

This is very common when you're working behind a corporate firewall or proxy that uses SSL inspection (man-in-the-middle decryption).

I've struggled with this problem in my company assigned machine, and below steps are for your reference and hope they can solve issues on your side as well:

Since my system uses a custom root CA certificate, it is needed to manually import that certificate into the Java TrustStore used by Protégé.

1. Identify the Java Runtime

Protégé often comes with its own bundled Java Runtime Environment (JRE). You need to find where it's located:

- Look in the Protégé installation folder for a directory named `jre`.
- If it's not there, Protégé is likely using your system's default Java (check your `JAVA_HOME` environment variable).

2. Export the Corporate Root Certificate

1. Open your web browser (Chrome or Edge).
2. Go to the URL the registry is trying to hit (usually <https://protege.stanford.edu>).
3. Click the **Padlock icon** in the address bar -> **Connection is secure** -> **Certificate is valid**.
4. Go to the **Details** or **Certification Path** tab.
5. Select the **Root Certificate** (the one at the very top of the hierarchy).
6. Click **Export** or **Copy to File** and save it as a `.cer` or `.crt` file (e.g. `company_root.cer`).

3. Import the Certificate into the KeyStore

Open a Command Prompt or Terminal as **Administrator** and run the `keytool` command. You will need to point it to the `cacerts` file inside the Protégé JRE.

Below command assumes you're running in Windows machine.

```
keytool -import -trustcacerts -alias corporate-root -file  
"C:\path\to\company_root.cer" -keystore  
"C:\path\to\Protege\jre\lib\security\cacerts"
```

- **Default Password:** The default password for the Java keystore is `changeit`.
- **Trust:** When asked "Trust this certificate?", type `yes`.

4. Alternative: Manual Plugin Installation

If you can't bypass the proxy/SSL issues, you may download plugins manually:

1) Visit the "Protégé Plugin Library" (<https://protege.stanford.edu/ontologies.php>) in your browser.

- 2) Download the `.jar` file for the plugin you need.
- 3) Drop the file into the `plugins` folder inside your Protégé installation directory.
- 4) Restart Protégé.

Referencde

- Protégé Official Website: <https://protege.stanford.edu>
- Protégé Wiki: <https://protegewiki.stanford.edu>
- Michael DeBellis, *A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins*:
https://www.researchgate.net/publication/351037551_A_Practical_Guide_to_Building_OWL_Ontologies_Using_Protege_55_and_Plugins
- Protégé Pizza Repository: https://github.com/yasenstar/protege_pizza
- EKA Official Website: <https://xiaoqi.com/>

Demo Video for this Chapter

YouTube Demo Video - Chapter 03: <https://youtu.be/Q6eq-cWBpfQ>

Last updated at: 5/16/2026, 4:41:43 PM

Chapter 04 — Creating Classes and Building the Pizza Ontology Skeleton in Protégé

In the previous chapter, we explored the Protégé workspace, navigating the ontology editing environment and understanding its key panels and perspectives. We also began to develop the mindset of a semantic engineer -- thinking in terms of conceptual categories, machine-readable meaning, and structured knowledge.

Now, we move from understanding the tool to **actively modeling domain concepts**. This chapter focuses on the first practical step in ontology engineering: creating classes and defining a structured hierarchy. Classes are the core of OWL ontologies -- they represent abstract concepts in a domain, define the structure for semantic relationships, and form the foundation for reasoning and knowledge graph construction.

From the perspective of the **Executable Knowledge Architecture (EKA)**, creating classes represents the first concrete step in translating enterprise knowledge from **meta-models and diagrams** into **formal, machine-understandable semantics**. Classes are the backbone nodes of your knowledge model, linking conceptual design to the eventual executable intelligence layer.

By the end of this chapter, you will be able to:

- Understand the role of classes in ontology modeling
- Create root and subclass hierarchies in Protégé
- Apply meaningful annotations to each class
- Organize the Pizz ontology skeleton for scalability and reasoning
- Connect your class structures to the broader EKA framework
- [4.1 Understanding Classes in Ontology](#)

4.1 Understanding Classes in Ontology

In OWL, **classes** represent **conceptual categories** -- abstract groupings of things that share common characteristics. They answer the typical question: _"What types of entities exist in this domain?"

In the Pizza ontology:

- **Pizza** represents the overarching domain concepts for all pizzas.
- **Topping** represents ingredients or components added to pizzas.
- **VegetarianPizza** and **NonVegetarianPizza** represent specialized subclasses of **Pizza**, reflecting the dietary classification.
- **CheeseTopping**, **MeatTopping**, and **VegetableTopping** are subclasses of **Topping**, further organizing the ingredient concepts.

Classes are the **semantic anchors** for your ontology. They enable logical inheritance, reasoning, and relationships to be defined later.

In EKA terms, classes transform conceptual diagrams into **formal semantic nodes**, bridging the gap between enterprise knowledge design and executable intelligence.

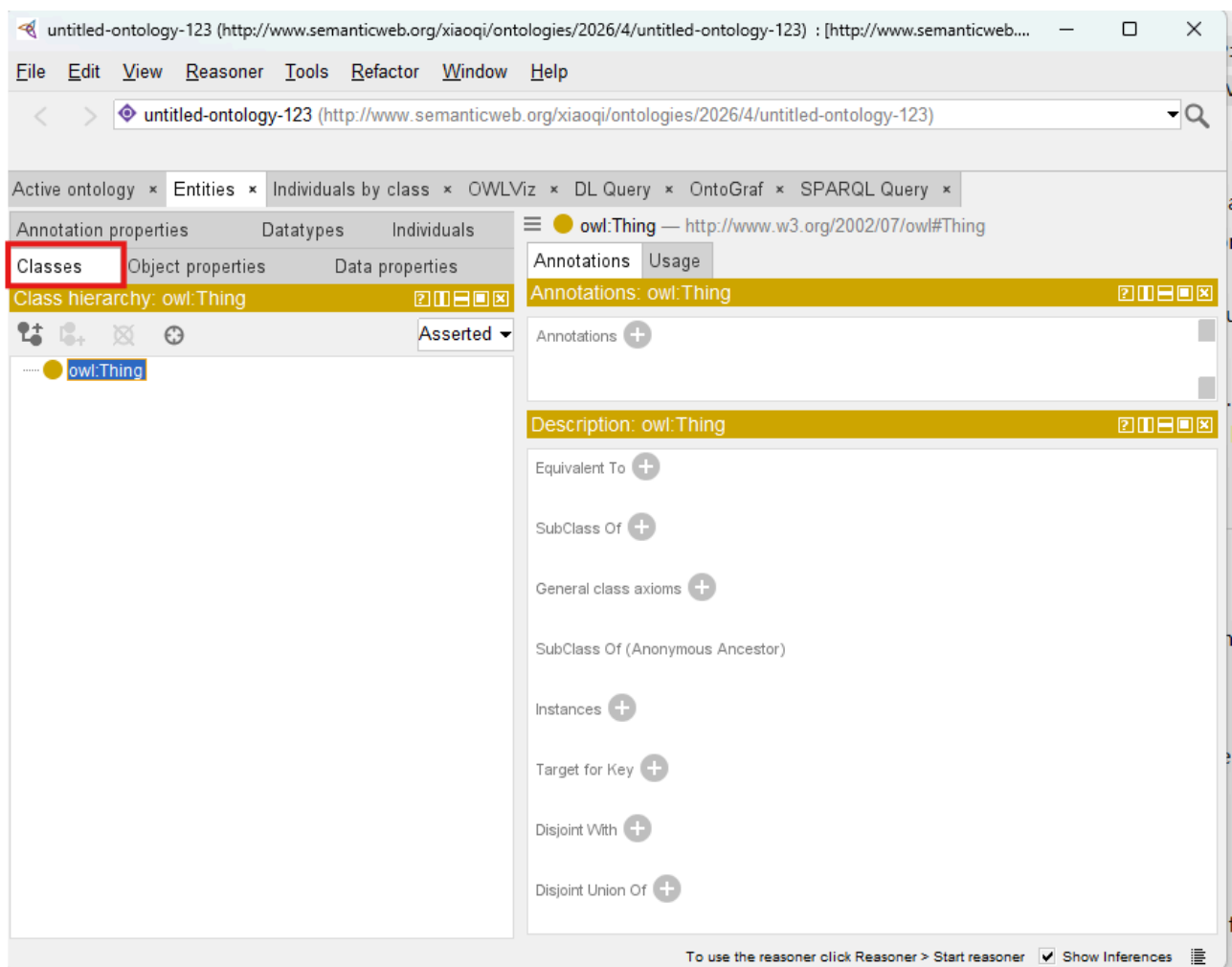
Without a solid class hierarchy, downstream ontology operations like object property assignment, instance creation, and reasoning can become inconsistent or unreliable.

4.2 Creating Classes in Protégé

The process (step) of creating classes in Protégé introduces you to **hands-on ontology engineering**:

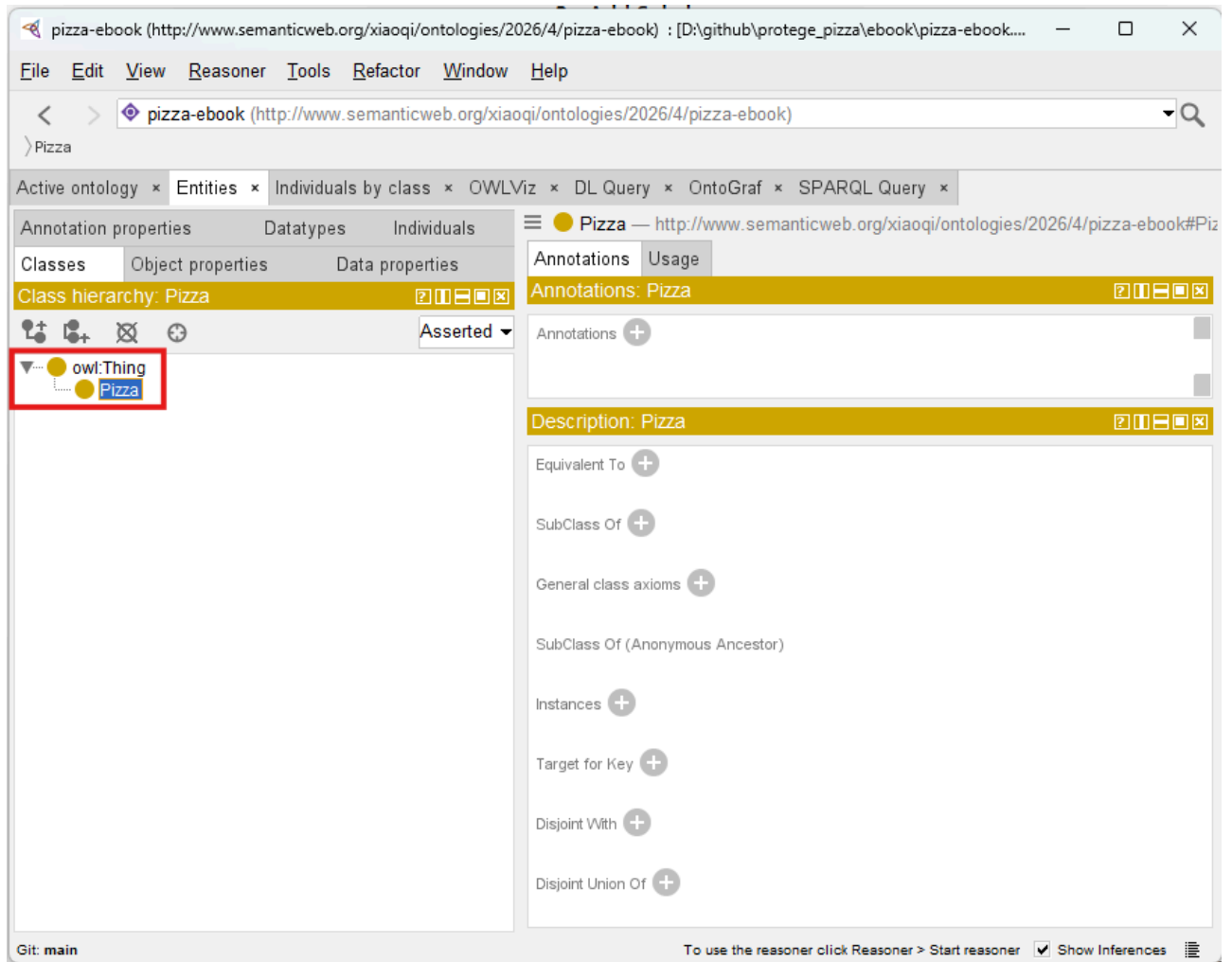
1. Open the Classes Tab:

Protégé organizes ontology classes hierarchically in this tab. This interface allows for creation, subclassing, and annotation management.



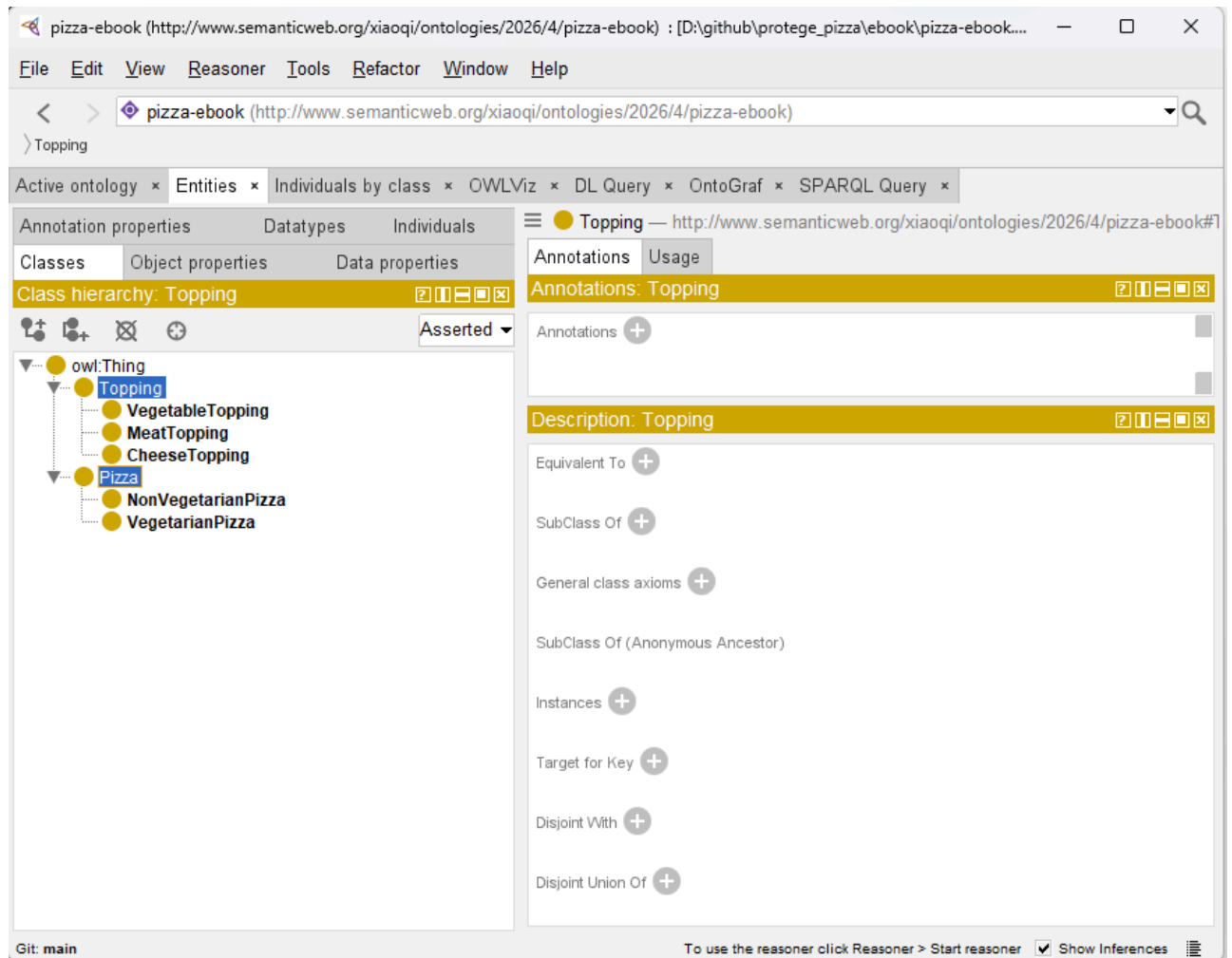
2. Create the Root Class:

While `owl:Thing` exists by default as the universal root, creating a domain-specific root, `Pizza`, gives semantic clarity and establishes a class starting point for the ontology.



3. Add Subclasses:

- Under **Pizza**, create **VegetarianPizza** and **NonVegetarianPizza**.
- Under **Topping**, create **CheeseTopping**, **MeatTopping**, and **VegetableTopping**



4. Manage the Hierarchy:

- Ensure logical "is-a" relationships.
- `VegetarianPizza ⊆ Pizza` means every vegetarian pizza is also a `Pizza`.
- Subclassing defines semantic inheritance, which is foundational for future reasoning.

5. Validate and Save:

Regularly saving and checking hierarchy consistency prevents structural errors. Protégé's visual hierarchy aids in spotting potential misclassifications.

This process demonstrates how **conceptual knowledge is formalized**. It also highlights the EKA principle that **well-structured ontology classes serve as the nodes of a future knowledge graph**, capable of supporting reasoning and executable intelligence workflows.

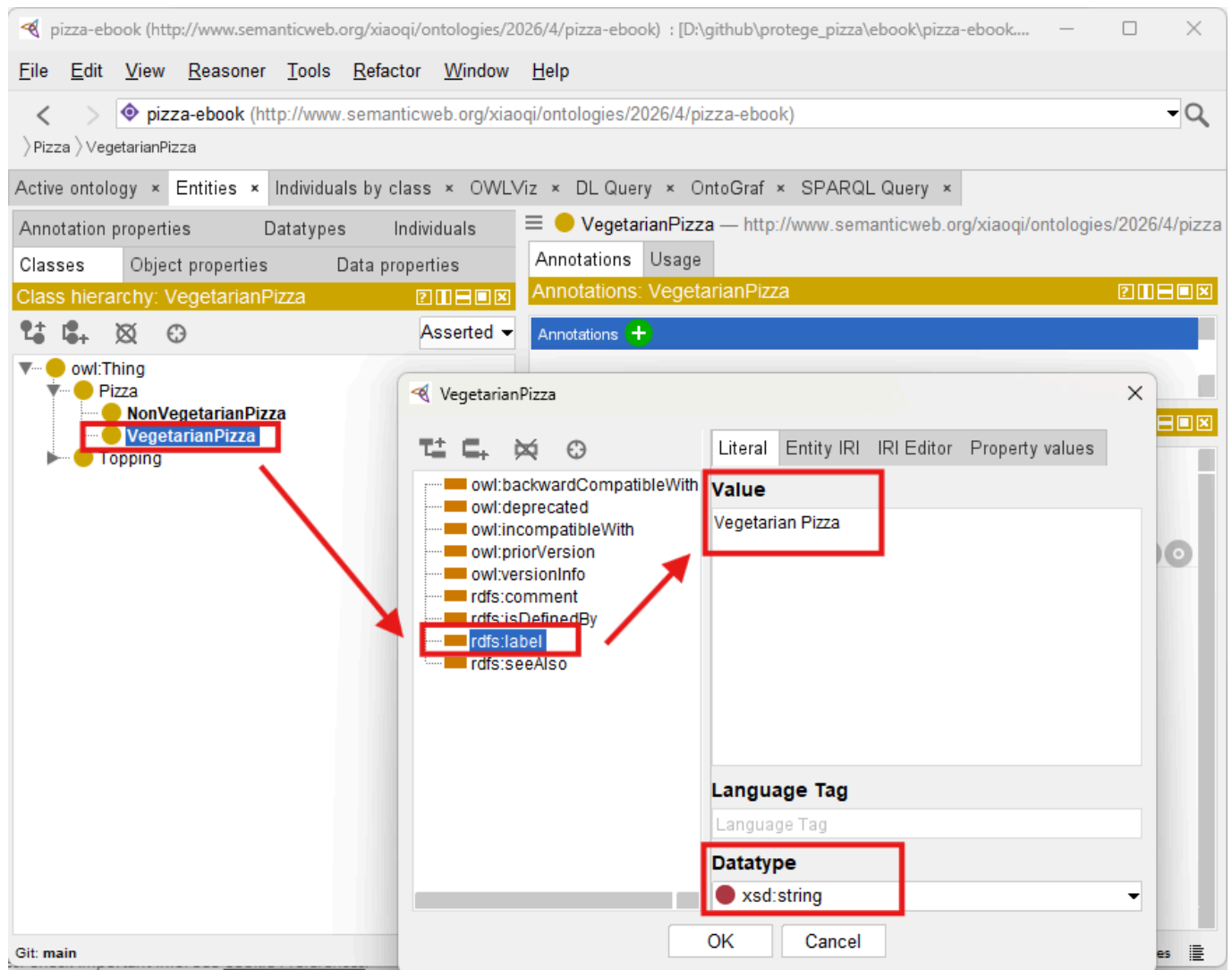
4.3 Adding Annotations

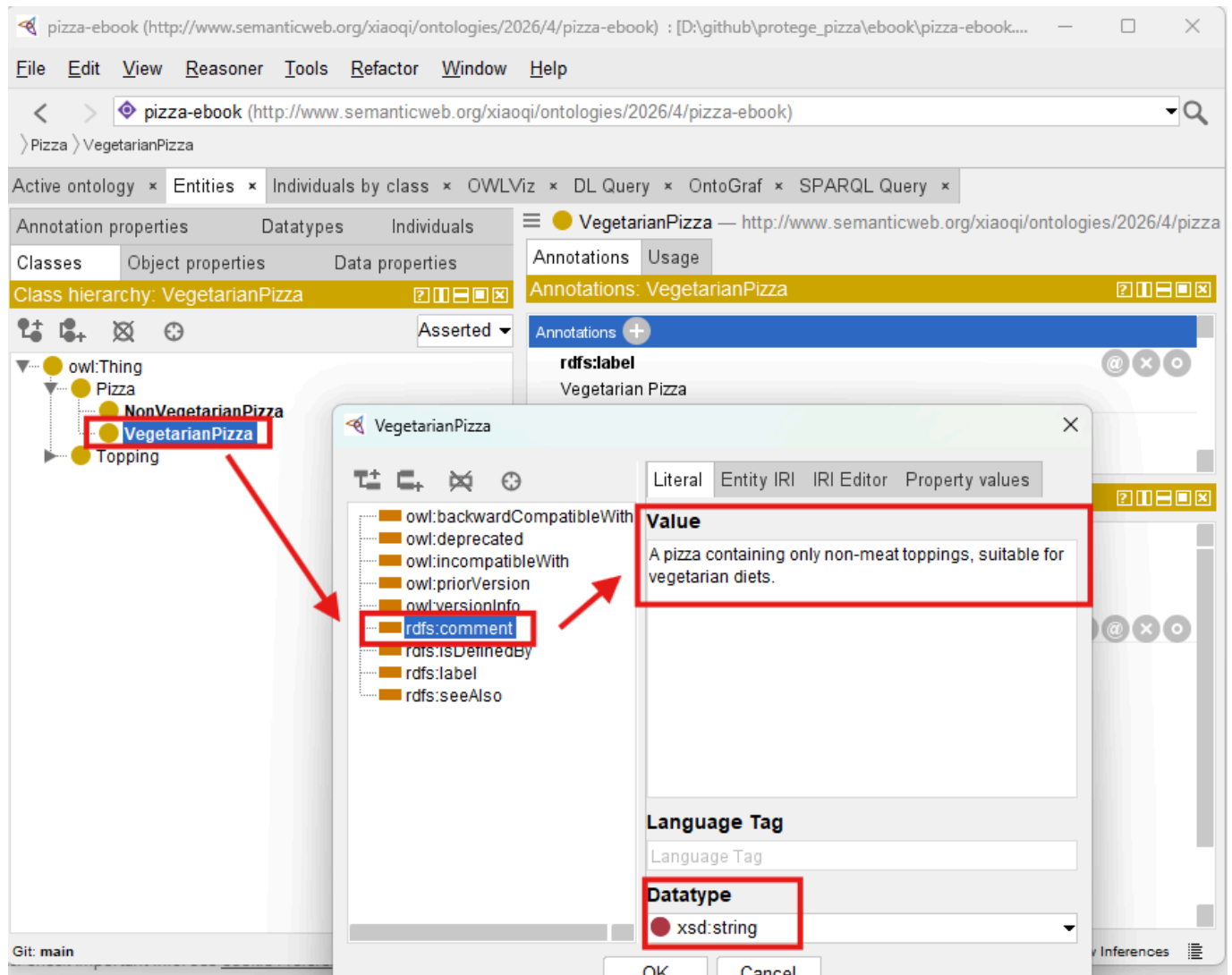
Annotations provide human-readable metadata for classes, making the ontology understandable to humans while remaining machine-readable.

- **Label** (`rdfs:label`): A description name of the class.
- **Comment** (`rdfs:comment`): An explanation of the class' purpose or semantic meaning.

For example:

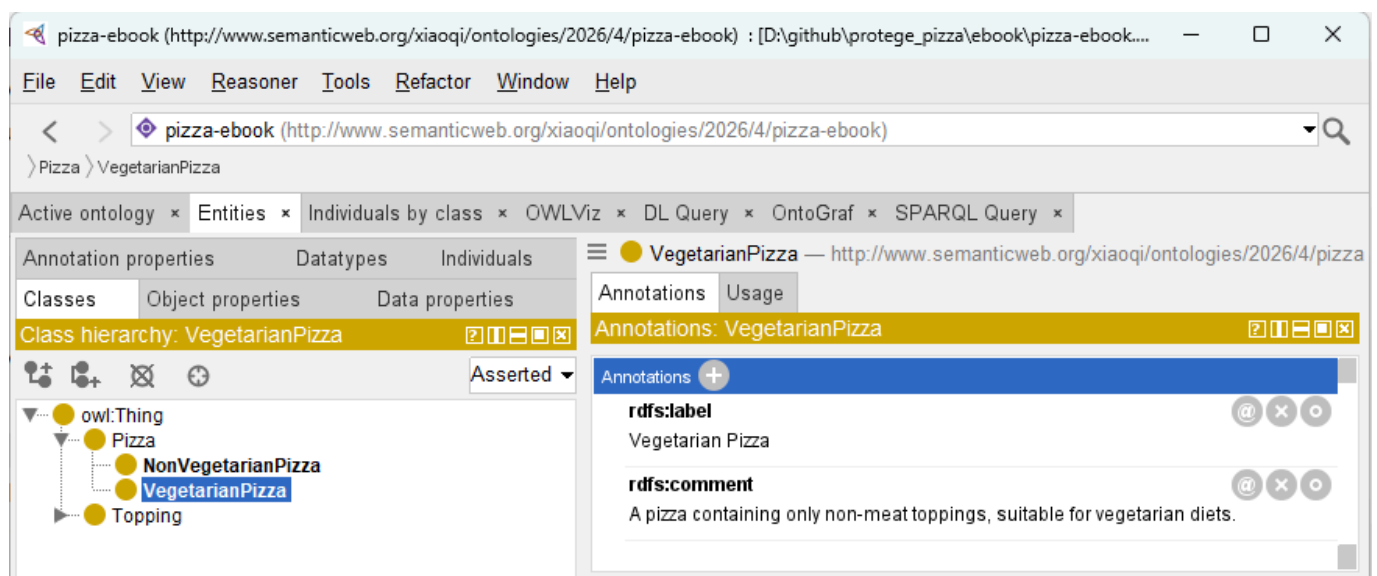
- Class: **VegetarianPizza**
- Label: "Vegetarian Pizza"
- Comment: "A pizza containing only non-meat toppings, suitable for vegetarian diets."





[!Note] You have to click **OK** after Label setting then add Comment annotation, everytime only one annotation can be set

After adding, two annotations are as below:



4.4 Organizing Subclass Hierarchies

Hierarchy is more than a visual tree -- it defines **semantic inheritance**, which is the foundation of reasoning:

- **VegetarianPizza** inherits all properties and constraints from **Pizza**.
- **CheeseTopping** inherit from **Topping**.

Best practices:

1. Maintain a clear root-to-leaf path to avoid confusion.
2. Avoid duplicate classes; semantic ambiguity undermines reasoning.
3. Use comments and labels to clarify distinctions.
4. Ensure consistency with your meta-model diagrams.

This step mirrors the **Diagramming** → **Meta-Model** → **Ontology** pathway in EKA. Your diagrams represent conceptual ideas, meta-models define the structural rules, and ontology classes encode these as executable semantic constructs.

4.5 Practical Exercise: Building the Skeleton Ontology

By the end of this chapter, you should have a **working skeleton ontology**:

- **Root Classes:** **Pizza**, **Topping**
- **Subclasses:** **VegetarianPizza**, **NonVegetarianPizza**, **CheeseTopping**, **MeatTopping**, **VegetableTopping**
- **Annotations:** Labels and comments for all classes
- **Clean Hierarchy:** Logical parent-child relationships aligned with semantic inheritance principles

This skeleton forms the base upon which object properties, individuals, and reasoning rules will later be applied. The exercise reinforces the **progressive semantic difficulty principle**, gradually preparing you for more complex modeling.

4.6 EKA Perspective

Creating classes is a **critical step in the EKA roadmap**:

1. **Diagramming:** Conceptual pizza categories visualized in diagrams.
2. **Meta-Model:** Defining rules and semantic categories for structured knowledge.
3. **Ontology:** Translating classes into machine-readable semantic nodes.
4. **Knowledge Graph:** Classes will serve as nodes connected by properties, forming the backbone of semantic reasoning.
5. **Executable Intelligence:** Accurate class structures ensure that reasoning engines can infer, validate, and execute knowledge.

By properly designing classes, you are establishing **semantic scaffolding** that allows enterprise knowledge to become executable and AI-ready, bridging traditional modeling with cutting-edge knowledge engineering.

Chapter Summary

In this chapter (04), you have:

- Explored the central role of classes in ontology modeling
- Created root and subclass hierarchies in Protégé
- Added annotations to improve human and machine interoperability

- Built the first skeleton of the Pizza ontology
- Connected practical exercises to EKA, understanding how classes become semantic nodes and eventually executable knowledge.

This exercise forms the essential foundation for future ontology enhancements, including object properties, individuals, and reasoning logic.

Key Concepts

Concept	Description
Class	A conceptual category representing domain entities.
Subclass	A specialization inheriting semantic properties.
Annotation	Human-readable metadata supporting clarity and documentation.
Hierarchy	Structured parent-child relationships defining semantic inheritance.
EKA Connection	Classes link diagrams and meta-models to knowledge graphs and executable intelligence.

Protégé Skills Learned

- Navigating the Classes tab
- Creating root and subclass hierarchies
- Adding labels and comments (annotations)
- Organizing semantic hierarchies for reasoning readiness

Next Chapter (05) Preview

In the next chapter, we will explore **Named Classes**, the building blocks of formal ontology definition in OWL.

You will learn how to:

- Define clear and semantically meaningful class names
- Organize class hierarchies for scalability and clarity
- Apply naming conventions to support reasoning and knowledge integration
- Begin understanding how naming decisions impact ontology maintenance and interoperability

This step continues the progressive construction of the Pizza ontology, transforming your skeleton into a **structured, well-defined semantic model**, ready for further enrichment and reasoning.

Demo Video for this Chapter (04)

YouTube Demo Video - Chapter 04: <https://youtu.be/IMjKcx93ens>

Last updated at: 5/16/2026, 4:41:10 PM